

Driver Drowsiness Detector

Hardware Project Evaluation

Charles Smelt

F331073

April 18, 2026

Contents

Introduction	3
1.1 Why Do This?	3
Research	4
2.1 Tiredness Identifiers	4
2.2 Driving Effected Factors	4
2.3 Legality of Using a Mobile Device	4
2.4 Potential Models and Algorithms	5
2.4.1 Initial thought process – basic Convolutional Neural Netwrk (CNN):	5
2.4.2 Deep Convolutional Models:	5
2.4.3 Xception Model (Extreme Inception):	5
2.4.4 CondenseNet:	6
2.4.5 Further Development Ideas / Key Takeaways:	6
2.4.6 Information Communication:	7
Plan	9
3.1 Time Management:	9
3.2 Connections:	9
3.3 Component Choices and Reasoning:	10
Datasets	13
4.1 SIVAS-UNIVERSITY-OF-SCIENCE-AND-TECHNOLOGY-DRIVER-DROWSINESS-DATASET:	13
4.2 VICOMTECH:	13
4.3 FATIGUEVIEW:	13
4.4 BINA University:	14
4.5 DATASET POSITIVES AND NEGATIVES:	14
Development Checklist	16
Development	18
6.1 Setup:	18
6.2 Data Validation	18
6.3 Simple AI Models	20
6.3.1 Libraries	20
6.3.2 Simple AI Model	20
6.3.3 Inception Model	21
6.3.4 Dataset Resolution	22
Connectivity	24
7.1 Back-End:	24
7.2 Front-End:	27
7.3 Enabling on Boot:	29

Evaluation	31
8.1 Simulation Testing	31
8.2 Data Extraction:	32
8.3 Preliminary Observations	33
8.4 Quantitative Analysis	33
8.4.1 General Data Spread:	33
8.4.2 How Darkness affects the model's Prediction:	35
8.4.3 How well the mode performs over varying genders ?	37
8.4.4 Effect of accessories on accuracy	39
8.4.5 How does a user having their hair in their face affect the model?	40
8.4.6 Camera position effect on accuracy	42
8.5 Overall Accuracy	44
8.5.1 Distribution of Results:	45
8.5.2 Confidence in False Cases	46
8.5.3 Summary of Attribute Effects and Ideal Environment	46
8.6 Ideal Use Case / Environment:	48
8.7 User Feedback:	48
8.7.1 Areas of Success	48
8.7.2 Areas of Failure	49
8.8 Additional Remarks and Development ideas	49
Conclusion	50

Introduction

The aim of this project is to create a versatile piece of hardware that can be used to improve road safety. With use of a Raspberry Pi 4B and a camera (I have opted for a web cam as they are cheap and widely accessible), I will be aiming to produce a light weight AI based system to detect a drivers tiredness and aim to either wake them up, warn them of their condition or even potentially come up with a viable solution. I will further aim to inform the driver through use of a wireless digital display. For this, I am aiming to use the users own WIFI enabled device to connect to the Pi and broadcast data, warning as and other information. Since this will be presented to the user via a web page, I would further like the user to be able to interact with the system to Tailor it to themselves and their set up, to get the best function out of the project.

1.1 Why Do This?

Many accidents on the road are caused by drivers lack attention due to dreariness. With ‘10-20%’ [\[1\]](#) of all crashes on the road believed to have been attributed to tiredness; one in four fatalities estimated to have had tiredness play a part on UK roads, this is a serious problem. Not only this but given that an incident occurs on UK roads ‘Every 17 minutes’ [\[2\]](#) where ‘someone is killed or seriously injured’ [\[2\]](#), something needs to change.

Research

2.1 Tiredness Identifiers

There are several different ways to identify tiredness, especially when driving. Some examples are:

- More frequent blinking – this is because your eyes and brain function to counter fatigue and discomfort
- Harder Blinking – prolonged opening can result in them being dry, so, to combat this, your body makes you blink harder and more frequently to keep them lubricated
- Delayed reaction times – neurons in your brain slow as they process information less efficiently
- Your head can begin to drop as you inevitably slowly fall asleep (worst case)
- Yawning – this is your body's attempt to both keep your brain awake as well as cool the body
- Grip loosens – your body begins to relax as you fall asleep, actions aren't made with the same intent
- Mood changes – you can often become more irritable as you fall asleep
- Snoring – if you have already fallen asleep, you relax and this can result in snoring

2.2 Driving Effectuated Factors

As you get more tired, you often lose concentration. You can't find yourself reacting more slowly and your decision making can become impaired. You can also become a more violent or unstable driver as your mood can be easily influenced as you do not have the energy to combat how you feel. Not only this, but in the worst case (in the event of falling asleep) you can no longer operate the vehicle.

2.3 Legality of Using a Mobile Device

In this project, I would like to make use of the users' own device, which will probably be their mobile phone. To get around the legal issue of using a mobile device, it will have to be securely fastened in a cradle and not held by the driver. Furthermore, I will not be able to request interaction from the driver once the vehicle is in motion unless I use hands-free [6], "for example:

- A Bluetooth headset
- Voice commands

- A dashboard holder or mat
- A windscreen mount
- A built-in satnav”

Failure to comply with this can result in 6 penalty points and a £200 fine. But this can go up to a driving ban as well as a maximum of £1000 fine.

Key Implementation Takeaways:

- **Setup Protocol:** User-interactive features must be restricted to the initial setup phase while the vehicle is stationary.
- **Motion Safety:** Integration of the device’s internal accelerometer to disable interactive displays during motion, ensuring the system does not become a distraction.

2.4 Potential Models and Algorithms

2.4.1 Initial thought process – basic Convolutional Neural Netwrk (CNN):

My initial idea was to follow a methodology I had previously learnt in one of my prior modules, using a simple Convolutional model consisting of input nodes containing extracted values from an image, and then train the model to detect these trends in data to predict an outcome. For this I made use of Mediapipe’s face mesh to extract features from frames extracted from videos, to form inputs into the neural network. This was my initial thought process, and with a desire to start coding, and deliver a prototype, I naively started to develop this without additional research.

This methodology and implementation is talked about in ‘Simple AI Models’ section of this document. Here I explain my methodology and thought process (as well as data validation) as to why I believed this method feasible in the early stages of this project.

Following this failure, which is also expressed in the ‘Simple AI Models’ section I expanded my research to other AI models which will follow.

2.4.2 Deep Convolutional Models:

As an expansion from the idea above, this consists of a far greater number of layers. With its main uses in image identification, this would be ideal for the task at hand. One key difference being the number of layers. This adds size and complexity to the network, which may make it more difficult to run on the Pi compared to other models due to its increased parameter size as a result of its increases in width and levels.

2.4.3 Xception Model (Extreme Inception):

A development of the normal inception model, this stands as the middle ground between regular convolutional models and depthwise separable convolutional models (depthwise

+ pointwise). This makes better use of parameters than the Inception v3 model rather than reducing the number of parameters. This is done through channel mixing.

By removing the multi branch capabilities of an inception model, opting for a single branch this completely decouple spatial correlations as well as branch correlations, which results in more focused learning .

To take this further, you can make use of pre-trained models. The example I have used in the Inception Models section of this paper was trained on the ImageNet dataset. This transferred learning allows for pre-learns identification of edges, shapes and facial structure to be applied to my current data, without the need for the network to necessarily learn these itself, rather re-enforce its previous learning with my new data. This should allow for a model which learns faster, requiring less time to train, as well as a more accurate model, due to its additional pre-training. Some information gathered from [9].

2.4.4 CondenseNet:

This is an example of an “Efficient DenseNet using Learned Group Convolutions”[10]. This was designed for mobile use, with the aim of requiring limited computational power by removing connections between layers, aiming to remove redundancies which are not used in the later layers. Due to this, only one tenth of the computation is required compared to the DenseNet (a Deep Convolutional Model where all layers are connected to every other layer, rather than one layer being connected to the next). Comparable to MobileNet (an Inception model approach which focuses on efficiency and power usage rather than accuracy) but requiring less computation. This is done through pruning in the early stages of learning to form a ‘sweet spot’ [10] in between sparsity and regularity. Essentially, group (cluster) sizes are reduced (e.g. $3 \times 3 \rightarrow 1 \times 1$) which results in less computation, but a loss in accuracy, and requirement of more grouping of inputs. The idea of this is to reduce the number of connections during learning, to improve computational speeds and re-enforce routes, at the cost of accuracy.

2.4.5 Further Development Ideas / Key Takeaways:

Model Selection On such a small number of classifications and training examples for each classification, CondenseNet may not be the best option for this project. While it is said to be faster and more optimizable for a device with limited computational ability and power (like the Raspberry Pi I am using), it requires a greater quantity of training data than I have, without crossing between sources, and for the sake of this project, I believe that this may be beyond the scope of what is required for success – but this may change as the project progresses. Xception, while slower, can provide greater accuracy given the training data that I have. This would be good for experimenting on what the greatest achievable accuracy of a model given my training data could be. Not only this, but this model is available in Tflite, so it is possible for the performance to be improved while on the Pi. While not necessary, this is a nice bonus and something worth considering. Documentation on this can be found at <https://keras.io/api/applications/xception/>.

Pre-train Options: To improve the versatility of these AI models, the images that are used for training should be altered to allow for a more versatile classification. Since users

will be using this at day and night, it would be beneficial to alter some of the training data (if it isn't already at day and night) by adjusting the **brightness** to simulate darkness or lower light conditions such as changes in cloud cover or going through a tunnel. ($\pm 15^\circ$)

The camera angle and position can also change depending on how the user sets up the system. While the location of the recordings cannot be changed in post, the **angle** can. This will help to provide some fixability in the model, to still allow for detection of tiredness, even if the set-up is not perpendicular to the driver, like in most of the recordings. ($\pm 20\%$)

Another issue is **zoom**. My hardware, as well as the hardware between the different training data sources, is different. As a result, my inputs may automatically be different based on camera zoom, and how the user positions their face in the focus region, so fluctuations in zoom/crop may be a thing to consider. ($0.9 \rightarrow 1.1$ zoom/crop)

2.4.6 Information Communication:

Webpage Hosting With this project being done in python, for ease of use and versatility of existing libraries for the AI section of this project, it would be best to utilize some existing python libraries for the webpage.

Flask and Alternate Frameworks Flask is a lightweight web framework used for web-based interfaces and API's. In the case of this project, it would be used to host the local webpage. This will allow for offline communication of the drowsiness state.

Tkinter and PyQt provide a graphical use interface (idea for what I need), however, have a far more limited flexibility as well as requiring a physically connected device which will further limit how the project will function and be operated as additional wiring will create clutter as the user does not use something which is found in the vehicle anyway. As a result, neither of these frameworks are likely to be considered in the final result.

Django is a very powerful framework, however, with this comes complexity for a system with such simple routing and visualization requirements.

Another example of another framework to consider is REACT. With past experience in other projects, this has the upside that I understand it already. However, with the project being aimed to be as light as possible to improve efficiency, this client-heavy framework might not be the best for this project.

Overall, the low resource requirements and lightweight nature of this project, as well as the majority of the code base being in python, the obvious choice seems to be Flask to provide a platform-independent web interface.

API A simple way of connecting the webpage to the AI model. There are several different ways of communicating data from the tiredness detection AI to the webpage:

- **Shared File:** The idea is that both programs for AI detection and web hosting are run simultaneously with a common file to relay information between them. However, this could be more compute heavy, something I have expressed repeatedly as being something I want to reduce. Additionally, this can result in race conditions, where there is inconsistency between the file and the actual value. From a hardware point

of view, with files being stored on the SD card (with limited numbers of writes) this method can result in shortfalls in the long run.

- **Shared Database:** Using some form of database (e.g. SQLite) can provide additional information as well as filtering. However, because of the simplistic nature of this project, it is likely too costly to do this. While given more time, and a greater fixation on tracking and suggested improvements to inform the user of patterns or issues, this is not in the scope of this project.
- **Use of integrated API:** This is the least expensive option in terms of computing power. You can make use of threading to keep the two lightweight, related programs, together. This makes it easier to manage and in the event of failure of the AI, the whole program knows (as opposed to being in the dark due to each program being separate).

Therefore, **Option 3** (combined server and AI runner with threading) was selected as it minimizes computational overhead, avoids disk I/O bottlenecks, and keeps the architecture simple for this real-time detection application.

Plan

In this section, I will outline a brief outline of my intended plan of action, which will likely be reformed later on as other issues become more present during the development process.

3.1 Time Management:

Driver Drowsiness Gantt Chart:

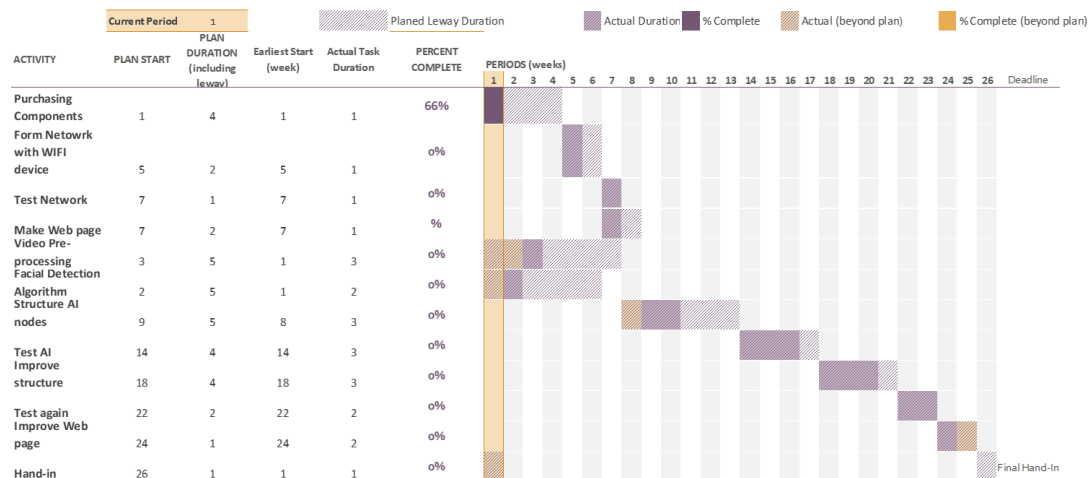


Figure 3.1: Initial Thoughts on a Project Pipeline

Here week one refers to the starting week (w/c 20.10.2025) and week 26 being the week of the deadline (w/c 20.04.2025). The initial delay in this project is due to shipping of components. Since I do not have any of the components that I need to do this project, I will have to order them, and this takes time. However, in the meantime, I can request permission for datasets as well as gather a better understanding of what attributes can affect a driver's concentration and different potential models that I could use for my lightweight AI component.

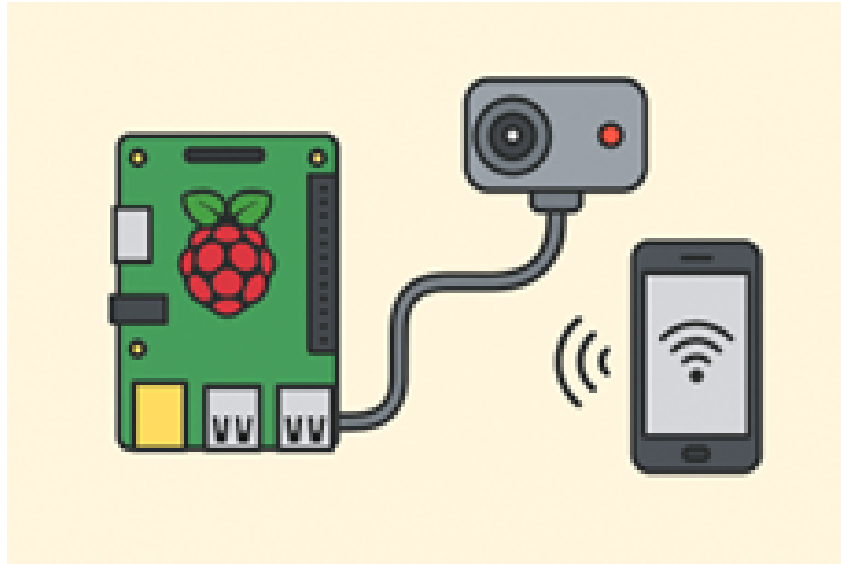
3.2 Connections:

Since raspberry pie 4 is WIFI compatible, I plan to connect the user's own device by this. Therefore, using the pi as a host device for a website that can display data and other information to the user. This is the ideal option as wired displays either require the pi to be:

- Dashboard mounted due to a small wire. This has the potential to fall or move, causing further irritation or distraction for the driver. This is against the aim of the project of keeping the driver focused on the road.
- More wires and additional screens for setting up. Once again, this takes up space in their field of vision and will create added hassle if setting up for different drivers.

Most people will likely opt to not use this device, making it a futile construction.

By using the users' own wireless device, you not only remove the added expense of the screen (if released to the commercial market) but also remove clutter. Users are likely to already have phone mounts in their desired location, so why not give the idle device some added functionality. This will also be optional, if they are already using the device for maps, it will not prevent the system from relaying information or alerting the user.



(ChatGPT generated -> prompt="please make this look nice + drawing of this")

3.3 Component Choices and Reasoning:

Raspberry Pi: For the main processing unit of this system, I have chosen a Raspberry Pi 4B. While this might be over the top in certain aspects, such as:

- Basic Graphics Processor
- 4 USB-A ports
- Ethernet capabilities
- HDMI compatibility
- GPIO Header (40-pin)
- Audio/Video jack

All of these are not necessary for this project specifically; however, I will be using this for a variety of additional projects beyond the scope of my dissertation. I chose model 4 because this has sufficient computation power to process video or images fast enough that it should not hinder the analysis. This should also be able to stay cool (while staying silent due to no fans) while running my intended AI model.

Camera: Initially, I wanted to use a rear dashcam as this was small and would provide the infra-red capabilities I desired for nighttime video, however this required additional wiring which would make this more expensive. In future, I would recommend a smaller

camera module, however, for the current constraints and the plug and play capabilities, this is the superior option. These also often support 30fps. This isn't too fast of a frame rate that you are unable to extract and process the data between the frames, as well as not being too slow that you miss vital things such as blinking (which can further be used to detect tiredness).

WIFI Capable Device: This is the area of the project that can have the most flexibility in requirements. I will be making use of several different devices that I own (old phones or tablets), with the intent of users being able to use whatever they have available.

Structure: This project can be broken down into the following areas of consideration:

- **Hardware:** Selection and configuration to allow the support of a real-time driver drowsiness detection system. This includes the use of a Raspberry Pi as the main processing unit, as well as peripheral devices to capture user data required for operation.
- **Data Pre-processing:** To ensure data quality and consistency, before training the models. The handling of missing or corrupted data should be performed, as well as general normalization and clear labeling to allow for an easy workflow.
- **Image Processing:** This is a core component of the system, enabling facial detection and analysis from still images (frames). This will consist of the following:
 - Facial Feature Extraction
 - Model Development
 - Training
 - Testing

Video Processing: Following the development of single frame processing, it is a good idea to apply this to a series of frames taken from a video input. This will allow for it to give the real-time analysis that is required. This stage will consist of:

- Extracting frames for reference
- Facial Feature Extraction
- Model development
- Training
- Testing

Following the development of the AI model, the next stage of the project is to display these findings and apply warning or other safety measures to the user.

Networking: Connecting to the user's device. Providing a connection to host a webpage on, and later allow for alerts and information to be passed

Webpage: Develop a simple webpage to display system outputs and status information.

Hosting: Host the webpage locally on the Raspberry Pi in a display-only configuration. This will help to ensure that the user does not interact with the system from within the vehicle. This will help to ensure safe operation in a vehicle setting.

Mobile Device Connectivity: Connect the user mobile device to the Raspberry Pi via local network connection to access the hosted webpage

Additional: To ensure safety and conform to laws, disable actions from the webpage (if any are sued) while in motion.

Datasets

For this project, I have used several different data sets from varying sources around the globe in order to try and train a more versatile system. From Turkey to China, all of these sources are publicly available and will be noted, referenced and explained in this section. Please note that additional references will be made through this document. I will outline where the datasets can be found, steps to access it for yourself as well as legal outlines that I must abide by in order to use them.

4.1 SIVAS-UNIVERSITY-OF-SCIENCE-AND-TECHNOLOGY DRIVER-DROWSINESS-DATASET:

Collected by a Turkish University, this data set contains 19 drivers (3 female and 16 male) who recorded themselves using their phones in a variety of positions. These are in .mp4 formats. [3] Their GitHub can be found at: <https://github.com/EsraKavalci/Sivas-University-of-Science-and-Technology-Driver-Drowsiness-Dataset>

4.2 VICOMTECH:

This organization has many different datasets available such as Body pose estimation and Hands-on-wheel detection. Following my research into fatigue identifiers. I first found out about this dataset through FatigueView [5] where it was referenced as an external data set. This dataset once again has a variety of users, this time with an average age of 30. IT also outlines additional features to consider, such as glasses wearers and other driving activities such as drinking water, on the phone and other actions which may be useful to better understand how drivers act differently when fatigued. Their Page: <https://dmd.vicomtech.org/> [4]

4.3 FATIGUEVIEW:

This is not only a dataset, but also an article on how tiredness affects drivers and what identifiers exist to help identify fatigue. This uses data collected from several different people, some many different backgrounds. This data set not only contains videos of people who are already tired but also shows how people can change from awake to drowsy as time goes on. This is done through a simulated driving environment with many cameras at varying angles to help capture the most versatile data. This will be useful, as the varying camera angles will allow the model to be more forgiving to change, allowing for an individua to place the camera in the best location for them in their own car. Their article: <https://ieeexplore.ieee.org/document/9931532/figures#figures> [5]

4.4 BINA University:

A dataset collected by a university in Indonesia, the idea of the study was to “hypothesizes that drowsiness can be accurately detected in real-time through computer vision analysis” [7]. This was collected using a Raspberry Pi 5 and a camera module, taking frames at a rate of 30 FPS. This dataset takes more of a face on positioning. This will allow for more accurate extraction of features for testing as turning of the head is less likely to result in the loss of information (where other datasets with more of a bias to one side will lose facially features when they turn away – the same change has a more dramatic effect). Furthermore, this dataset set also contains night recordings. This is important as you are:

- likely to be more tired at night
- contrasts in images and the visibility of attributes is likely to change (due to reduced lighting) so it is important to take this into account

<https://data.mendeley.com/datasets/chvz7vh2dc/1> [7]

4.5 DATASET POSITIVES AND NEGATIVES:

Camera Location: By using a variety of different camera angles, when training the model, it will be more lenient to change. Fixed locations can result in the data being more tailored, and so the AI model will be more tailored. This can influence reliability as it can result in the model being more specialized. Changes to location could potentially have a large impact on the output and its reliability. Varying countries can also have the same effect. Different countries drive on different sides of the road and therefore drive on different sides of the car.

Varying User Groups: Like location, varying users is key. Different ethnicities and languages can vastly improve data reliability. While some individuals are more expressive and others harder to identify features, this can help improve the versatility of the extracted data. While not something I would have initially considered, language spoken can play an effective role. Some languages are more articulatory and expressive so require more facial movements, which may be construed as yawning or signs of tiredness (especially if audio isn’t considered).

Varying Hardware: While my project intends to use consistent hardware throughout, the variation in hardware used during data collection across different datasets provides a degree of flexibility. Since my hardware doesn’t exactly match that of one or more datasets, having access to multiple datasets helps mitigate potential inaccuracies that could arise when testing on fixed data versus real-world usage scenarios.

varying Time: With some data sets spanning both night and day, there will be ranges in clarity and lighting. It can be a safe assumption that people (in most instances) are more tired at night; be this due to a long day or your circadian rhythm (your body’s internal clock). This may also vary the number of ‘Tired’ events in a given period, simply due to time of capture.

Bais: With less women taking part in the study (across all datasets) it is likely that it will be less accurate at detecting their tiredness. While the general indicators are the same, additional features such as longer hair which can distort imaging may play a part. Additionally, glasses (while represented in the dataset) might influence facial detection and lead to false results. This is something I will have to consider later. Other areas such as eye shape and size varying through individuals might make certain groups of people more difficult to identify tiredness, while others are more prone to different patterns for being tired. Time of day will also be a factor. It is easier to see people in the day (when they are more likely to not be tired). As a result, determining tiredness might not be as straight forward as the less clear data (relating most to the area I want to identify and train the model for) will be harder to extract information for. Not only this, with some datasets being collected from nighttime workers, they might be tired during the day, which will reduce the effectiveness of time within the data.

Development Checklist

In this section, I will be noting any tasks that are required to complete the project. While some smaller sections that posed no issues may be glossed over and only noted in the “Development” section, most of the step-by-step stages will be mentioned as a check list for someone to follow if they want to create this project for themselves.

Table 5.1: Project Task Overview

Task Ref	Task	Started	Completed	Status	Comments
T1	Purchase Components	–	29/10/2025	Complete	
T2	Face Identification	–	04/11/2025	Complete	In pre-captured images
T3	Taking frames	10/11/2025	11/11/2025	Complete	Able to take an image from the camera
T4	Extracting Features	20/10/2025	23/11/2025	Complete	Using MediaPipe to identify features
T5	Validate Data	07/11/2025	08/11/2025	Complete	Compare overlay on image
T6	Train Simple AI	08/11/2025	18/11/2025	Complete	Required additional preprocessing
T7	AI model on Pi	23/10/2025	25/11/2025	Complete	Lightweight model from Bina dataset
T8	AI model for Video testing	01/12/2025	03/12/2025	Abandoned	Inaccurate; buffer issues with model
T9	Data pre-processing	10/12/2025	19/12/2025	Complete	Prepared dataset for advanced model
T10	Advanced model training	20/12/2025	24/12/2025	Complete	
T11	Advanced model validation	26/12/2025	27/12/2025	Complete	Tested on validation dataset
T12	Advanced model testing (real world)	22/01/2026	22/01/2026	Complete	Tested with camera input
T13	Testing on Pi	–	–	Complete	tfllite model works with reduced runtime
T14	Set up connection between devices	01/02/2026	03/02/2026	Complete	Connection established
T15	Form basic webpage	02/02/2026	18/02/2026	Complete	basic shapes with no layout (yet)
T16	Advanced webpage design	17/03/2026	17/03/2026	Complete	just come css to make it look better
T17	Display webpage on device from Pi	05/02/2016	14/02/2026	Complete	Simple testing with random data
T18	API testing	15/02/2026	18/02/2026	Complete	With randomly generated date

Task Ref	Task	Started	Completed	Status	Comments
T19	Linking API and AI model	21/02/2026	03/03/2026	Complete	AI model used to generate data
T20	Starting on boot	08/03/2026	16/03/2026	Complete	Setting up a myscript.service file (used to take recordings, later re-purposed for the API by chnaging file path)
T21	Data capture	20/03/2026	27/03/2026	Complete	Gather some data using custom cap.py file and gather resources to evaluate
T22	Evaluating the model	30/03/2026	02/04/2026	Complete	Extract data from captured videos
T23	Statistical Analysis	03/04/2026	15/04/2026	Complete	Patterns and trends identified and commented on
T24	Getting user feedback	17/04/2026	18/04/2026	InProgress	General ffeedback on a complete system

finish filling in

Development

6.1 Setup:

Python Versions On my laptop (the main device for coding) I am making use of Python 3.10.5. This is because, mediapipe is not currently supported by the later versions that are available(at this time 3/1/25 it is supported up to 3.11.0). While the version that is installed by default on the pi is 3.13.5, I will be reducing this down to the same 3.10.5 to ensure all libraries work. To ensure there are no conflict, I will be using a virtual environment to run my python scripts in the correct version.

PI OS Setup I am using Debian with desktop for my Pi OS version as this is the most UpToDate (and recommended for my device). I am using the desktop version as this will allow me to more easily change code rather than using a straight Command Line interface, however, it would be possible, and I could ssh into the Pi to execute code from there.

Initially, I did try the command-line approach, however, issues in the connection timing out and for the sake of convenience, I thought it was best to make use of a desktop OS version.

Face Detection I am using the libraries OpenCV (Open-Source Computer Vision) and Mediapipe to detect the face and extract data. OpenCV allows me to open the camera, read a frame and detect the existence of faces. After this, I am making use of a Mediapipe library to extract facial details. Using some of the built in features these libraries provide, I then overall a triangular mesh which I can use to extract key points on the face, such as:

- Eye openness
- Mouth position
- Nose and chin locations to determine orientation / head tilt

Together, I can take video and images, and then extract the information required for me to determine drowsiness.

6.2 Data Validation

This section is designed to break down the spread of events in a data set and ensure that there is a difference in the data, so it is viable for training some form of AI.

SUST Dataset This refers to the ivas-University-of-Science-and-Technology-Driver-Drowsiness-Dataset [3] This shows an example of 5 videos for each classification (chosen at random). This is using a 2fps sample rate.

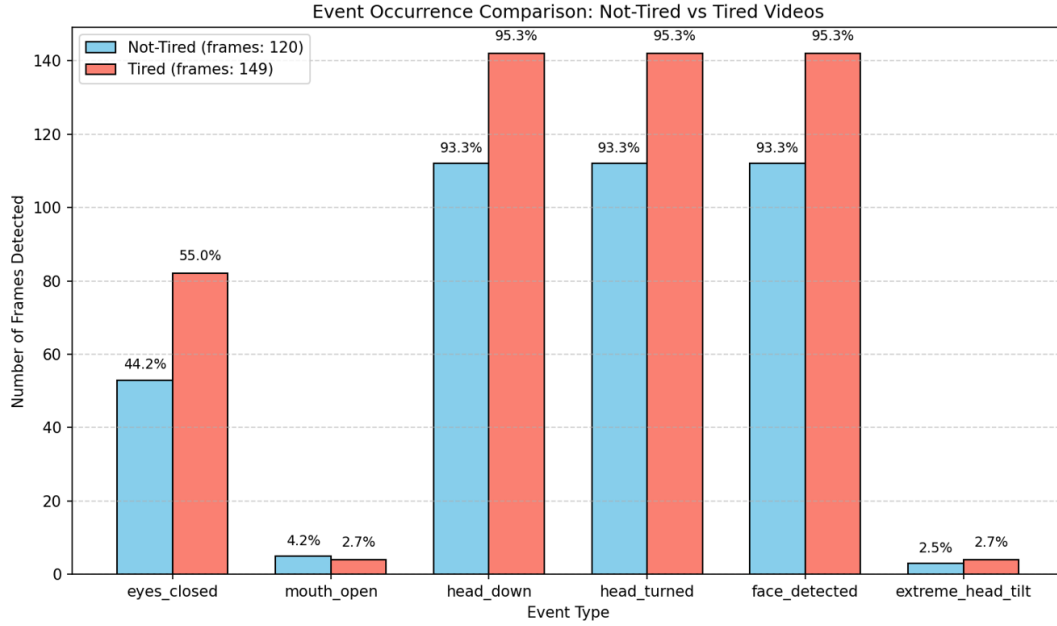


Figure 6.1: Distribution of events in the SUST[3] dataset

As you can see, there is a distinct difference between the 2 datasets, and so this is viable for development of an AI model. However, surprisingly (could be just from the chosen videos) that there seems to be reduced mouth movement when tired. This was not an expected result!

Something else that caught my eye was the number of **extreme_head_tilt** events. By exploring different threshold values, I was able to determine that while tired, you tilt your head more (physically) than when awake.

However, following on from this, I formed a lightweight model where I passed in a selection of 5/6 videos of each classification and attempted to see if these events alone were enough to form a valid model. This was a failure. This is because despite there being frames which contain these tiredness events, most of the video was populated with frames which showed no attributes of tiredness (despite tiredness events, a majority of the ‘normal’ frames were classified as tiredness). This resulted in a poor model which had very low confidence in its classification (52%). Therefore, this is not a viable strategy.

Bina Nusantara University This dataset (as discussed earlier) is a collection of images, which are labeled with “eyes closed”, “eyes open”, “yawning” and “not yawning”. These labels are purely from the pre-made labels file that comes with the data; however, I wanted to be sure that there was a range of instances for both the tired and not tired counterparts. This suggests that there is enough variation in the dataset to potentially form an AI model.

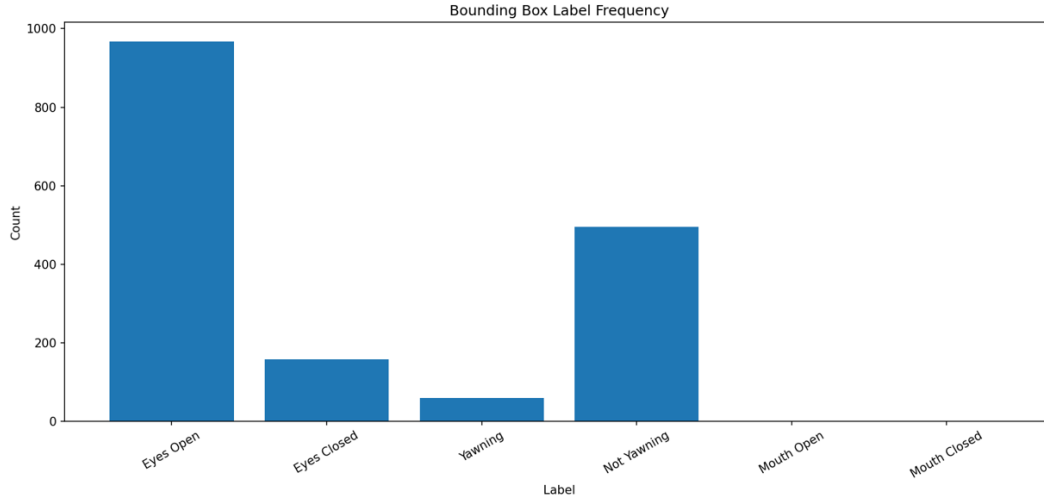


Figure 6.2: Distribution of calssifications within the BINA[7] dataset

6.3 Simple AI Models

6.3.1 Libraries

TensorFlow Easy to use and customize. One thing I have done to help improve the computability of the project was do most of the training and coding on my laptop, then importing the project from my Git Repository onto my pi, allowing me to train models and test more easily, rather than trying to do it directly on the pi. Using TensorFlow, I am easily able to save models as '.h5' file and then run them later.

TensorFlow Lite For ease of use and in the hope of improving running time, I will be converting my standard '.h5' models into '.tflite' models. This is because general tf models are designed for training as well as inference, while tflite models are aimed more at just passing data through them. In the case of a Pi, this more specified computation will help to improve the return rate of the model/service, allowing me to run it more in a given time. This is done by compressing the models, reducing the required RAM and using a C++ runtime as opposed to the normal (more cumbersome) python.

6.3.2 Simple AI Model

Based on my initial findings of what constitutes tiredness, I thought it best to extract the features that determine tiredness and input their ratio values into a simple CNN model.

The model that I have used so far is comprised of 5 inputs [left eye aspect ratio, right eye aspect ratio, mouth aspect ratio, pitch, yaw] extracted from the frame. This is then assed into a convolutional model consisting of 2 hidden layers of size 32 and 16 respectively. This uses the 'Adam' (Adaptive Moment Estimation) optimiser method which adjusts weights combing the benefits of momentum and RMSProp. On each layer the model makes use of ReLU (Rectified Linear Unit) as an activation function, allowing positive values to pass through and negative ones to change to 0.

From this simple model, I was able to produce deduce tiredness at an accuracy of 74.5%, based on frames extracted from the Bina Nusantara University dataset. I believe this

is because the classification of eyes open compared to eyes closed, where some examples of training data (as well as test data) were miss labelled from the source (so it is worth while to change these like I did to ensure the data is classified correctly for training). This improved accuracy slightly to just over 76%.

Following this, I thought it was a good idea to test its compatibility on the Pi and use myself as a test subject. Initially, I thought this was working brilliantly, as closing my eyes or imitating a yawn would result in the classification of Tired (with a similar amount of accuracy to on the testing data). However, as my testing became more extensive – testing with varied facial expressions and testing the effect of pitch and roll on classifications – I noticed a discrepancy in model predicition. As I tilted my head to the left (moving my ear towards my shoulder), no matter the facial exression, the model predicted **tiredness**. This is something that was completely overlooked in the testing data and implies there is a subtle trend in the data that your head tilted left is a sign of being tired. This is a spurious correlation and could be down to camera position.

My initial thought was to retrain the model as it may be that that classifier (through that run of training) has gotten to a local maximum (locally high bias) and been unable to recover. However, despite other iterations, I was unable to produce a more accurate model, fluctuating between 68% - 76% accuracy on training data, with no more luck on testing on myself and a webcam.

From here, I thought it best to experiment with a different model to see what the upper bound in obtainable accuracy was.

6.3.3 Inception Model

This will be used to determine an upper-bound on the potential accuracy a model can give. With its primary purpose, to identify images, this is ideal. As stated in the research section of this document, I will be proceeding with the Xception Model to classify images. The initial model I used was pre-trained on the ImageNet dataset. This will allow for the easy identification of features (e.g. lines and shapes) without needing for the model to learn them from scratch, which should help to improve learning rate and require less data inputs to create a fine-tuned model. From here, I trained it on the Bina University [7] dataset as it was composed of just images and contained a label file which would make classification easier.

Issues: Following the training of the model, the outputs as to classifications were in the same order as they were in the labels file. As an oversite on my part, I did not translate these labels (in the file) to English as (from the overview of the project) I could do it later. However, during my testing, I found some of the predicted labels were not lining up with my perception of the images. For example, I though an image was someone with their eyes closed, but the labels said otherwise. I went through I found that the dataset only had 3 classifiers in the labels, not the 4 that I had been told. From what I could understand, yawning and not yawning were translated properly, however the eyes open/closed labels were substituted inn for a neutral face label, which has the same translation as eyes open (when converted to English).

Solution: To fix this issue, I wrote a short section of code (based on my initial feature detection process from the data validation section) to turn these classifications into valid labels (only for eye openness). Following this, I then went through each image, which had changed, and verified the labeling. Following this, I then re-trained the model on these new labels. Then I tested it on the validation dataset, resulting in 131/137 (95.6% accuracy) images being correctly classified. With those which were wrongly classified, I went through and had a look at where the model was wrong. In 4 out of the 6 total images which expressed false-positives for eyes closed, I could understand why the model had perceived the people as having their eyes closed. In the other 2 examples, one person was wearing sunglasses (which unsurprisingly) caused an issue, with only 1 being truly wrong, classifying as eyes closed, when they were open, as well as yawning when their mouth was only slightly open.

Based on these findings, I then wanted to check how the model worked on me as a use-case. To my disappointment, it was not as successful as the validation would suggest with only around a 70% success rate based on my attempts to fake tiredness. While I understand that my acting abilities are not perfect, I was very surprised to find that I was not able to get a result in a similar ballpark, given that fact my expressions were likely over-exaggerated. Additionally, when looking back through my code, in my attempt to push a working model out before the exam season began, I have noticed that I had not altered or included the variations to input data, like I had in the Simple Convolutional AI models. As expected, after adding in the variations to:

- Brightness
- Contrast
- Saturation
- Flipping the image

,the accuracy of the model has decreased (relative to the testing set), with the hope that it is more versatile for a wider variety of end implementations. With a 75.91% accuracy (104/137) on the Bina University dataset resulting in correct predictions, this was more of a detriment than I predicted.

6.3.4 Dataset Resolution

While the previous model was reasonably accurate on determining if the user had their eyes open/closed or if you were yawning or not, it would require an additional threshold to determine tiredness from this. In my earlier model versions, I determined the classifications of eyes closed or yawning as a positive case of tiredness. However, with the Xception model taking in image inputs, I wanted to experiment with opinionated learning. This would allow me to define the dataset based on my opinion of someone looking tired and considering a larger array of features such as:

- Eyes open (but not fully), implying they are drained
- Bags under the eyes
- Mouth positions for talking vs yawning

,with this in mind, I decided to go through manually re-classifying the data and re-train several models on this. One of the things to consider when doing this is opinion. Since I am classifying the data, my opinion may be different to someone else, which can lead to inconsistencies based on person classifying because of a larger array of external factors. With this being an opinion and not necessarily a definitive solution yes or no, there was a slight reduction in accuracy to 73.72% (only a 2.19% loss compared to the previous model which used definitive classifications); far better than I could have hoped, for with an 85.33% accuracy in detecting ‘alertness’.

Personal Testing: Following my previous tests on myself, I found that I would try and remove personal bias to a favorable model by formulating a quantitative dataset that I can test against. While only consisting of 100 images and a fixed environment, I wanted to test model robustness thorough varying facial expressions. Despite using a webcam and in my room with no natural light and unlikely camera placement (vastly different from the actual use case), I was still able to obtain a 63.27% accuracy in detecting ‘tiredness’. From the results, however, all errors were False Positives for alertness (mistaking the user for being alert compared to tired). This could be down to a variety of factors:

- **Camera Placement:** With the camera being mounted ontop of my monitor (standard webcam location), it was directly in the line-of-sight. This minute change could result in my eyes being more correctly orientated for an Alert classification.
- **Lighting:** To reduce discrepancy caused by glare, I aimed to produce a high contrast image by removing background light and to reduce the effect of external factors. However, this lack of ambient light and harsh gradients could have formed an environment that lies outside of the training distribution.

Following on from this, it would be more ideal to generate testing data in the correct working environment to get a more accurate representation of the models predictive capabilities. Additionally, the effect of background lighting and camera placement might also be affecting the model and so a more extensive and varied dataset might be more representative of overall performance.

Connectivity

In the following section I will discuss how the classification will be presented to the user.

Overview As previously outlined in the design section, I would like for the information to be displayed as a webpage hosted on the Pi. This will require several components:

- Web page: for the actual displaying of information
- API: for the sending and receiving of the information form the AI model to the user
- WIFI Hosting: to allow the user to connect to the device in a dynamic setting rather than using a common network which may or may not exist in a movable setting such as a car

7.1 Back-End:

Network Architecture and Hosting By utilizing Raspberry Pi OS (Desktop), I can easily toggle between “Hotspot Mode” for device testing and “WI-FI Client Mode” for pulling updates via Git. This flexibility of dual purpose drastically accelerated the development and testing cycle of this section of the project, with the ease of not needing to type commands or form a toggle file.

Initial Flask Implementation To validate the connection, I developed a light-weight Flask application to serve a basic index.html file. This contained no information, images or other resources, just text. This confirmed that devices could successfully connect to the Pi’s hotspot and access the web server by inputting the Pi’s IP address. This application was constructed to specifically use `port 80` (HTTP), removing the need to define the connection protocols when handling IP addresses (Such as when changing the hostname)

Local DNS Resolution To provide a seamless user experience, I wanted to remove the IP-based navigation and instead navigate with a custom hostname. To do this I used `dnsmasq`. Using this, I configured the domain “**drowsiness**” directly to the local IP address for the application.

Performance Optimisers (DNS Upstream) Since the system operates as an offline connection, with no external internet access, I disabled recursive DNS lookups¹. This improves the performance by preventing the Pi from attempting to contact upstream servers for un-known queries, eliminating the latency caused by waiting for timeout responses. However, with this in mind, it is important that this change is reverted when connecting to the wider internet as this can prevent you from accessing resources such as the GitHub repository. To revert simply remove “no-resolv” from `/etc/dnsmasq.conf`. This will allow the device to now look towards up-stream providers in the event a webpage cannot be found.

¹the process where a device tries to ask other (more senior) servers for corresponding IP addresses for addresses outside of its local scope/knowledge

```

no-resolv
no-poll
interface=wlan0
bind-dynamic
address=/drowsiness/10.42.0.1
dhcp-range=10.42.0.2,10.42.0.254,255.255.255.0,24h
# Configuration file for dnsmasq.

```

Listing 7.1: dnsmasq Configuration

Table 7.1: Explanation of dnsmasq Configuration Parameters

Configuration Line	Functional Explanation
no-resolv	Tells the system not to look at the system's <code>/etc/resolv.conf</code> file, preventing it from using the ISP's upstream DNS servers.
no-poll	Stops the service from checking for changes in the upstream DNS configuration files. While not strictly necessary, it helps maintain system cleanliness.
bind-dynamic	Safety net to help ensure robustness. Used to link an IP address to an interface when the service starts. This is because the dnsmasq can be ready before the hardware, so the moment the it is ready, DNS and DHCP requests can happen.
interface=wlan0	Restricts the service to listen only for requests on the Wi-Fi interface. This allows the Ethernet connection to remain independent for interacting with the GitHub repository.
address=/drowsiness/10.42.0.1	Acts as a local DNS redirect for the 'drowsiness' domain, pointing it directly to the application's IP.
dhcp-range=...,24h	Defines the pool of valid IP addresses a connecting device can be assigned. This lease lasts for 24 hours.

Multi-User Scalability To allow for multiple devices to connect at once, I configured a /24 subnet mask to manage IP assignments. While the Raspberry Pi's hardware typically supports 5-10 devices connected concurrently, this ensures that:

- Multiple passengers can connect without causing IP conflicts
- The driver's connection remains stable, ensuring that safety critical data is always accessible, even if other devices accidentally "auto-join" the network

Reverse Proxy To further the reliability and security of the device, a **Rexerse Proxy** - (Nginx) ² is used. Then, to resolve the resource race between the **NetworkManager** and **dnsmasq**, the internal DNS capabilities of the NetworkManager were disabled.

```
[main]
dns=none
system-resolved=false
```

Listing 7.2: NetworkManager.conf modifications for preventing inbuilt DNS

This ensures that dnsmasq has exclusive control over domain name resolution and IP management, forming a more stable environment.

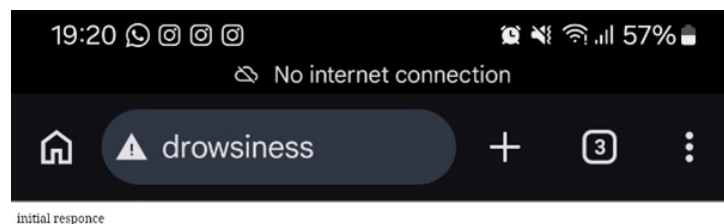


Figure 7.1: This is a Simple example of how a webpage URL can be inputted to make it easier for the user to find the resource. In this example, while 'drowsiness' worked on one browser, 'drowsiness/' had to be used on another as it assumes it is a search not a URL.

Captive Portal One of the other features that would be ideal is the use of a Captive Portal. Often, when connecting to public or guest Wi-Fi networks (such as a coffee shop or cinema) you must go through a page to accept their terms of conditions and potentially log-in. While I will not be using a log-in system, agreeing to terms and notifying the user of issues using the phone may cause would be a good moral stead, as well as it then easily allowing the user to interface/view the webpage after acknowledging the potential risk. This will be done by using DNS hijacking.

more about this

API As previously mentioned, I have opted for an integrated API and AI model solution, where the model work to use in-memory state management rather than a database or file; the main reasons being to reduce complexity by avoiding I/O overhead provided from databases and avoid race conditions.

Structure The API file will be broken into 3 sections for simplicity and ease of use:

- **Flask Initialization** To ensure the system is "headless" and user-ready upon booting, the Flask application is initialized within the main execution script. While the web server and the core logic could be decoupled, integrating them here ensures

²This is a layer of protection between the applicaiton and users which aims to protect the server (Raspberry Pi)

that all dependencies are functional before the system enters its operational state. Furthermore, this centralized initialization allows for the asynchronous loading of resource-heavy components, such as the AI model, ensuring the web interface is responsive while the backend completes its setup.

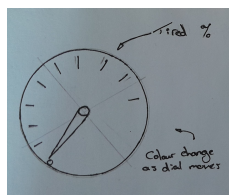
- **Prediction Generation** To maintain a real-time "safety-critical" stream, the AI model runs on a dedicated background thread. This decoupling is vital for multi-user support; without threading, independent API calls from multiple web clients would block the main execution loop, causing the model to drop frames and increasing system latency. To prevent race conditions during data exchange between the inference engine and the API endpoints, `threading.Lock()` is implemented. This ensures thread-safe access to shared variables, allowing one process to update the prediction data while others read it without data corruption.
- **Web Call Processing** The system utilizes relative URL routing for API requests rather than hardcoded IP addresses, which ensures the interface remains functional regardless of the network environment or the specific IP assigned to the Raspberry Pi.

7.2 Front-End:

Webpage Design The primary objective of the front-end interface is to communicate the user's predicted alertness efficiently while maintaining a discrete aesthetic. To ensure the interface is non-intrusive during general use – regardless of ambient lighting – a permanent dark-mode theme will be implemented. This will minimize user distraction and reduce eye strain during both day and night time use.

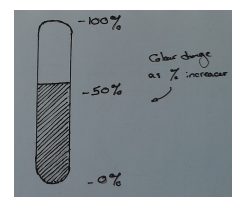
Design Ideas This section will give some examples of potential ideas. Some of these ideas will not make it into the final product, but may be considered in other related projects or in the event this project is repeated.

Circular Counter:



- + **Automotive Influence:** Helps the interface blend in and reduces driver distraction.
- **Implementation:** Complexity is higher due to formatting constraints

Biological Meter:



- + Improved clarity.
- + Easy to implement.
- Less aesthetically appealing.

Note: Similar to a fuel gauge already found in a vehicle.

Potential Implementations:

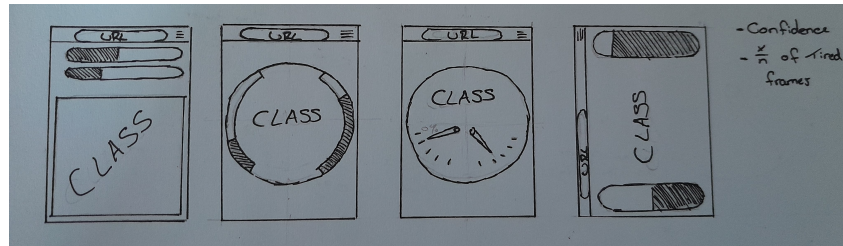


Figure 7.2: Overview of full display layout concepts.

Each of the designs above has its own pros and cons. The ideal interface should blend in with the surrounding vehicle to avoid distracting the driver while clearly conveying data. As a result, I have opted to combine the leftmost and rightmost layouts to ensure clear communication and simple implementation.

Integration Following the construction of a basic frontend, I wanted to see how it would look as a changing display – using random variable generation to get a better view of the system. To do this, I simply used the ‘random’ function to generate appropriate (possible) values and store them in the shared variables to be sent upon API call. Once information was successfully passed to the users device and could be viewed to trigger webpage items re-loading; the AI was added to be the source of the data rather than random data generation.

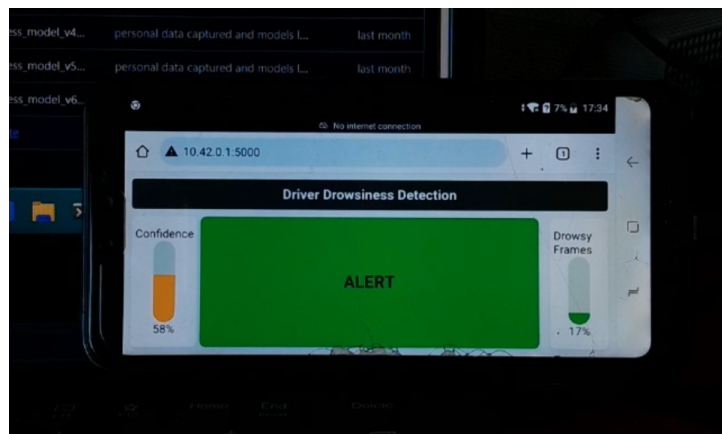


Figure 7.3: Example of integrated AI model Integration

Package and Data-Transfer Cache-Control is required to ensure that the system remains up-to-date and safety is paramount. This is done by teh following Modern Header Controlls:

- **no-cache:** forces the webpage receiving/requesting the data to check with the server before using the cached version of the data
- **no-store:** tells teh browser to **never** save any part ofm the request or response to the local disk
- **must-revalidata** reinforces the no-cache. Tells teh browser that once data becomes 'stale' (imediately) it can no longer be used

To help support older browsers (in the event the user has an older device), Legacy Headers are also used:

- **Pragma: no-cache** Check with the server before using the cached version of the data
- **Expired: 0** sets an expiration date to the beginning of the time count (01/01/9070), since all dates are now after that, it means the data will be expired instantly

To support this integration, the project was organized into a structured directory on the Raspberry Pi (Figure 7.4). This layout separates the core Python logic and Flask API (`api.py`) from the frontend assets (`templates/`) and the pre-trained weights of the Xception model (`model/`). By housing the `.tflite` model in a dedicated directory, the system can efficiently load the AI weights into memory during the server's initialization, replacing the initial random data generation with real-time inference results.

To allow for the testing of multiple different AI models and varying webpages, I decided to

```

/project-root
├── api.py .....python logic & Flask server
├── templates/
│   └── index.html .....HTML file
└── model/
    └── AI_model.tflite .....The AI model files
        (xception_drowsiness_model_v6)

```

Figure 7.4: Project Directory Structure on the Raspberry Pi

7.3 Enabling on Boot:

Since this is to be a 'headless' system, it is important for the program to run automatically (e.g. when the vehicle is turned on). Since general cigarette lighter power cables provide the 5V necessary for the Pi to function as intended, this can be used to power the system, however, without a display (thus far), you cannot enable the program. To rectify this, a **.service** file will be used. This has three sections:

- **[Unit]** - Acts as the metadata and dependency layer. It provides a description for debugging during status checks and defines the boot order. In this case, it includes a **soft dependency** (`Wants=network-online.target`) to ensure the system attempts to establish a connection before the script runs, while also ensuring the script only starts once the system is fully initialized.
- **[Service]** - This is the "execution" layer that defines *how* the script runs. It specifies the command to execute the Python script, the working directory, and the specific user permissions to ensure the script does not run with unnecessary root privileges. Crucially for a headless system, it handles **error recovery** via the **Restart** parameter, which automatically re-launches the program if it crashes or fails. I choose a delay period of 10-seconds because it allows me to rectify any issues (e.g. the camera becoming unplugged)

- **[Install]** - This section handles the "integration" of the service into the system's boot sequence. By using the `multi-user.target`, it tells the Raspberry Pi to link this service to the standard startup routine. This ensures that as soon as the Pi receives power from the vehicle and reaches its normal operating state, the service is triggered without any manual user intervention. This helps to ensure that the safe normal start-up occurs correctly before the program is run.

Evaluation

8.1 Simulation Testing

The best way to test the accuracy of the finished product is in the desired market location (in a vehicle). To collect the data, I established a simple program that will capture 5-minute videos of the course of an hour while you drive. Additionally, other datasets, such as the SUST [3] and BINA [7] will be used to help increase the variety and test range for the model.

The duration of the videos is not important, but I thought it was a good idea to break an extended recording session down into sub-sessions, in case removing power mid recording resulted in its corruption. I was hoping this would not be an issue (which it wasn't), but based on my past experiences with varying dashcameras, it was a risk I was not willing to take. From here, frames were extracted at regular intervals. These were then manually classified based on my own opinion of tiredness (the same opinion that the model was trained on). Then these frames were passed through the model. All of this data was then combined into a single dataset with the following attributes:

- **Actual classification** This is the classification I assigned to the image based on my own opinion of tiredness.
- **Predicted classification** This is the classification that the model assigned to the image.
- **Darkness** This is a value between 0 and 1 that represents how dark the image is. This was calculated by taking the average pixel value across the entire image and dividing it by 255 (the maximum pixel value).
- **Gender** This is the gender of the driver in the image.
- **Camera position** The position of the camera relative to the driving position.
- **Accessories** This is a binary value that represents whether the driver is wearing any accessories (e.g. glasses, hats, masks).
- **Hair in face** A binary value that represents whether the driver has any hair in their face (e.g. fringes, beards).
- **Alert_Confidence** This is a direct output from the model and represents how sure the model is that they are **alert** (not tired)
- **Tired_Confidence** This is a direct output from the model and represents how sure the model is that they are **tired**.

This will then be used to form a basis for my **Stratified Stress Testing**. Overall, I would like to see how each factor effects the model as well as how well the model general performs in everyday simulations.

8.2 Data Extraction:

How many of each classification? To determine if there is a trend between the predicted result and that attribute(s) it should fulfill the following:

$$E = z \cdot \sqrt{\frac{p(1-p)}{n}} \quad (8.1)$$

To achieve the ‘scientific gold standard’ of a 95% confidence that a pattern exists within the data, I have taken $p=0.5$ (the worst-case probability, equal to 50/50 a guess), n =number of images and E being the margin or error. Given this you need 384 images to provide a confidence of 95% within the data, and so, to cover all bases, I chose to aim for at least 400 images of each attribute (at a minimum). This can help reduce Type II errors by ensuring that there is sufficient evidence of this hypothesis.

How data was extracted? While simple and not necessarily relevant for the overall project, this was formed using the ***** library. This searched through a selected folder and displayed every image within it; any videos were dissected into frames (captured 5 seconds apart). From the videos, the frames were stored in temporary files to allow me to navigate through while removing the delay of having to search through a video for an image every time I wanted to move on (or to previous frames in the event of misclassification). Once this has been done, I manually went through every image and applied labels to each feature. While certain aspects such as darkness would be done automatically, determining the camera position relative to the driver would have been more challenging (simply taking camera position to relative head direction, would not be as informative as position as for an articular user, this is fixed following setup).

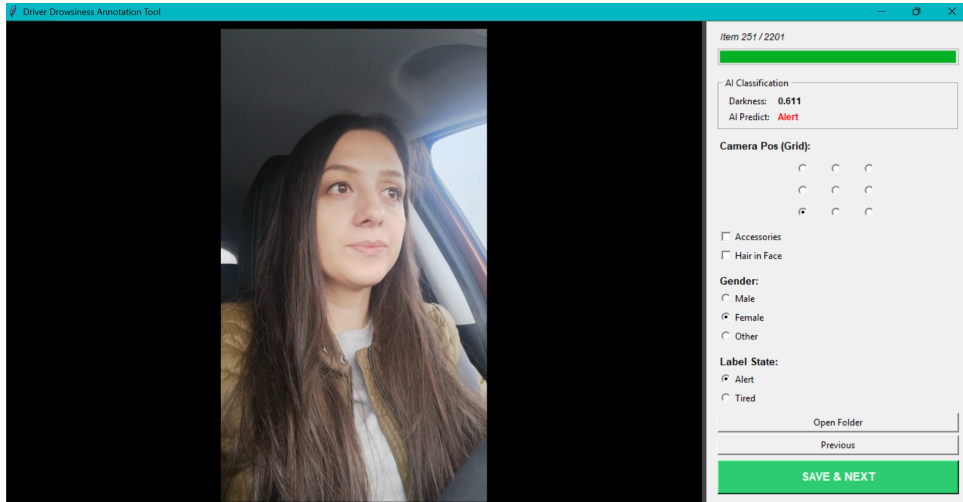


Figure 8.1: The interface for data extraction. Features an image from the SUST dataset [3]

All this combined into a 5676-line CSV file which would hopefully provide an insight into positive or negative trends within the model.

8.3 Preliminary Observations

While assigning variables, I noticed several trends in the data that I would like to point out prior to the quantitative data.

With the camera position being in a high-up position, the data tended to miss classify more than in other positions. I believe this to be because of how your eyes are orientated. With the camera being in a higher position, the eyelids are perceived as being more closed. This bias could potentially be represented in the data.

Eye shape. Some of the data contained people whose eyes are considered more of a ‘hunter’ eye. For these people, it seemed that the miss classification of tiredness was more prevalent than other people. This and other factors also appear to lead to one person having a higher tendency to one classification over another.

Also, for those who would sit more upright (or laid back) in their seats also seemed to be mislabeled more. This also seemed true for incredibly sunny environments for the same reason; they wouldn’t need their eyes as open to be able to see and so were considered tired incorrectly.

Another strange observation was that when someone was wearing a mask (even just under the chin) the person was often considered to be tired incorrectly, compared to other accessory features.

8.4 Quantitative Analysis

For this, I am using Orange [11]. This is a datamining software that helps identify features in both a visual and numerical way. This is used in industry for rapid prototyping, data visualization and exploratory data analysis.

8.4.1 General Data Spread:

For this I used a ‘column statistics’ node to express how each attribute is distributed over the entire dataset.

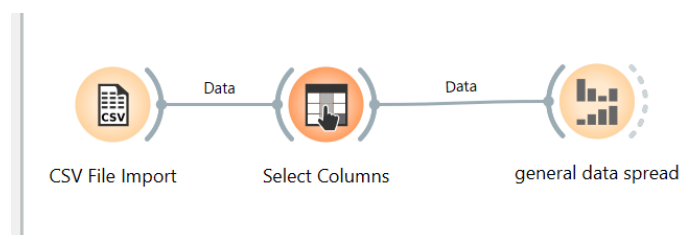


Figure 8.2: Node structure for general data spread

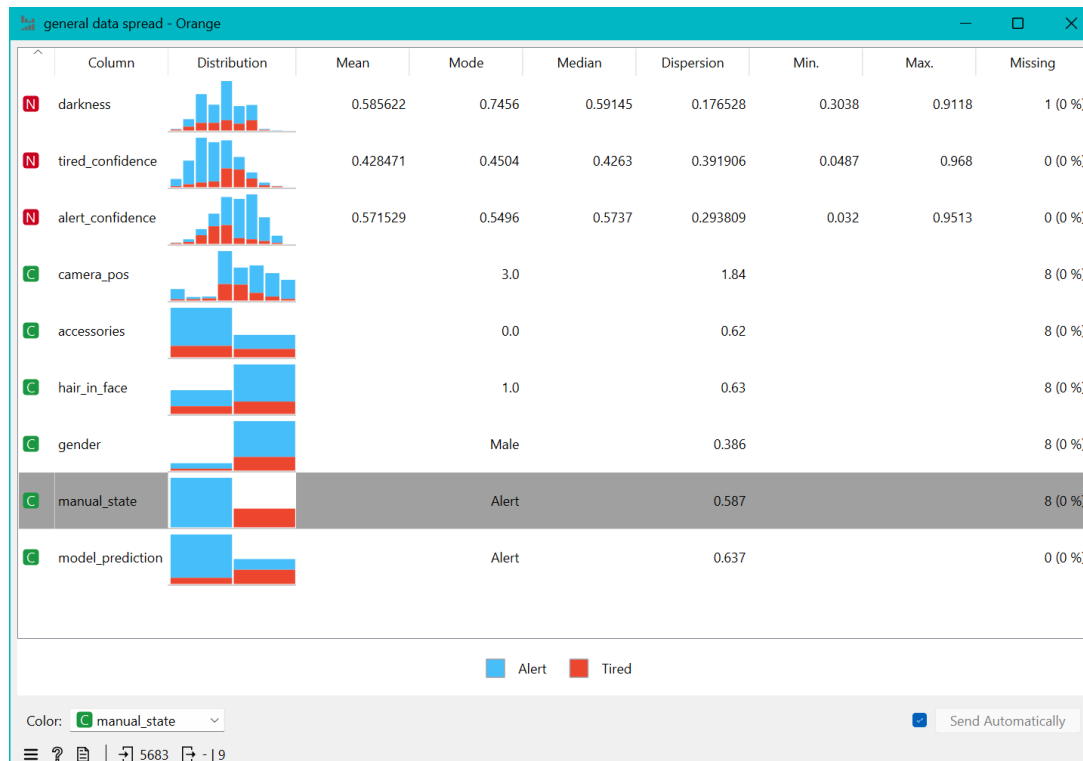


Figure 8.3: General spread of attributes within the evaluation data coloured by classification

There is very little data represented by the high center and high right camera positions. This could lead to my initial observation that higher positions are not as accurate being drowned out by the other data. There is a surprising amount of hair_in_face data. This is because beards and fringes were both counted as hair_in_face, the main dividing factor being the gender. Females with hair_in_face represents fringes or their hair down, while males with this attribute talk about beards or the appearance of stubble.

The main observation is that overall accuracy is very good. With 58.7% of the manually classified data being Alert and 63.9% of the models' classifications also being Alert, this implies there is a strong consistency between the model and my own opinions. However, you can also see from this that there is considerable discrepancy in my own classifications and that of the mode. But, despite this, the model appears to make mistakes more in the middle of the range (when it is more unsure) suggesting that there are no extreme paths within the model that have been allowed to spiral out of control (e.g. always classifying tired or always classifying alert). This is a good sign as it suggests that the model is not overfitted to one classification, despite the slight bias in the training data.

8.4.2 How Darkness affects the model's Prediction:

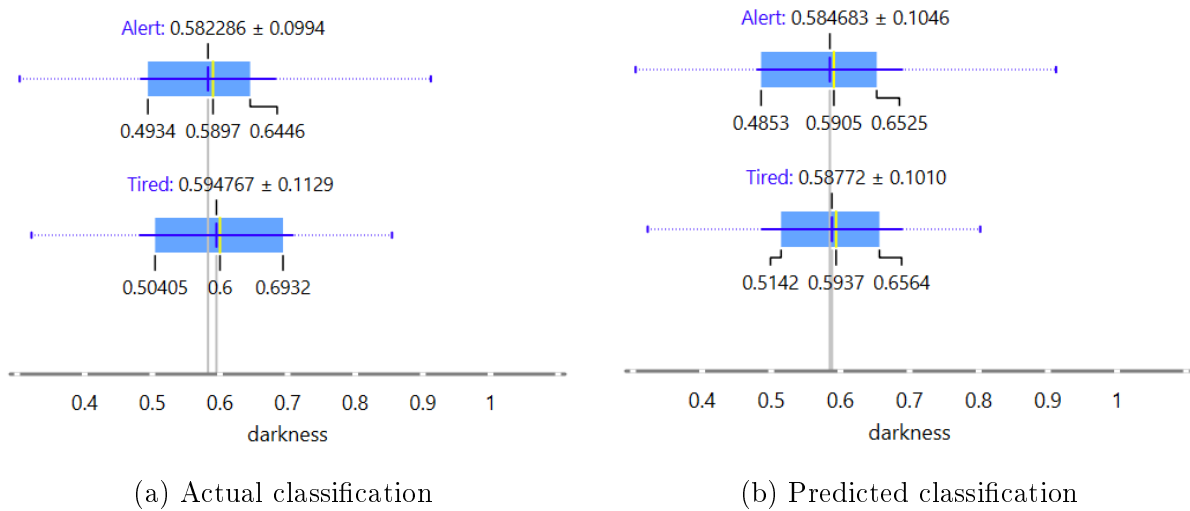


Figure 8.4: Comparison of darkness variance between actual and predicted classifications.

With the position of the median shifting towards lower values, it could be safe to assume that the model appears to be become less reliable in lower light conditions (especially when predicting tiredness). With the upper and lower quartiles also being smaller for the predicted values, it suggests that the model is more likely to detect tiredness in lower light conditions (which is to be expected as lower light implies earlier in the morning/later in the day – when someone is more likely to be tired).

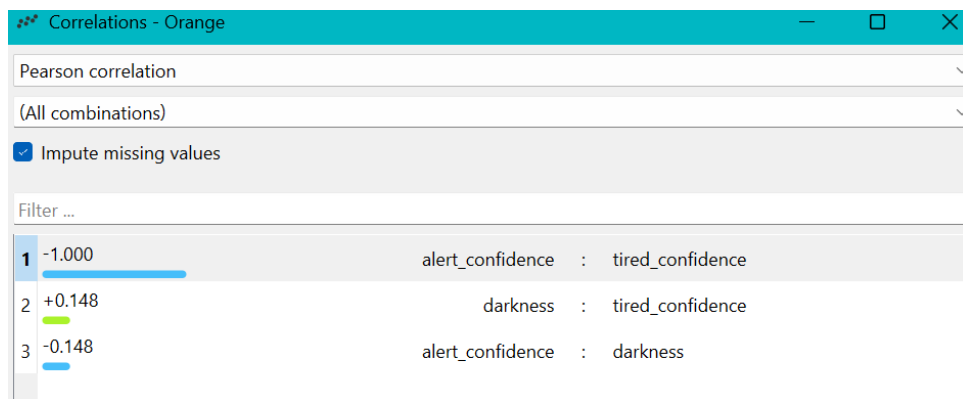


Figure 8.5: Darkness effect correlation to model confidence

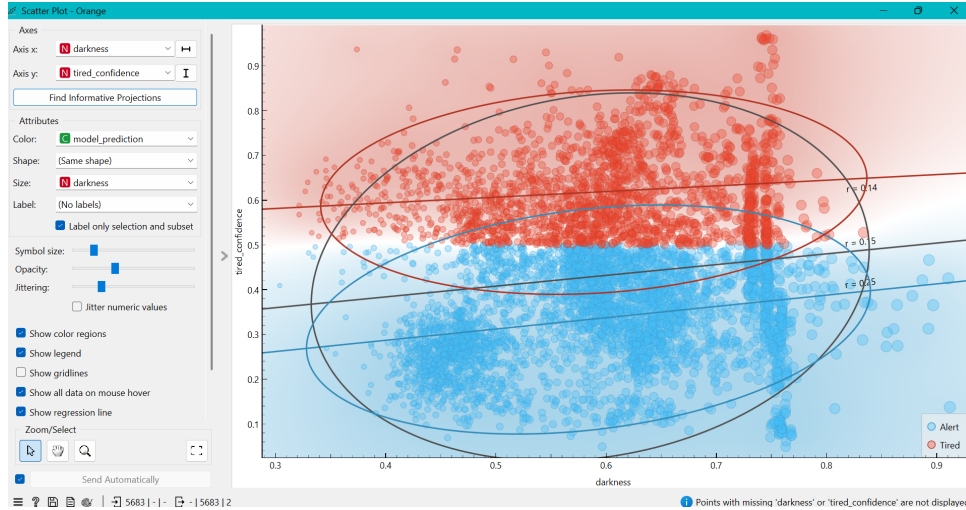


Figure 8.6: Darkness variation vs model confidence of tiredness

With the R values (regression lines) in (figure 8.6) being relatively low (< 0.3) but positive, suggests that there is a weak, but present, correlation between darkness and the model's confidence. This shows that there is a subconscious tendency within the model to predict a higher confidence for tiredness in lower light (when the image is darker) conditions, despite the user presenting as **alert**.

The lower correlation (R) between darkness and confidence for tiredness suggests the model is more robust to lighting changes. While **alert** predictions are more sensitive to light levels—potentially due to visibility issues or over-exposure causing subjects to squint—the model remains more consistent in its confidence across varying light conditions when identifying fatigue.

Observations on Data Density: Within Figure 8.6, certain vertical clusters of data points are visible, particularly around a darkness level of 0.75. Rather than representing a model bias, this high density indicates:

- **Batch Consistency:** Large subsets of the testing data were likely captured under identical environmental conditions, such as the same vehicle interior, specific time of day, or consistent camera sensor settings (or entire camera).
- **Discrete Metadata:** These "strips" represent the limitations of the darkness metadata, where multiple images are assigned the exact same darkness value despite slight visual variances.

8.4.3 How well the mode performs over varying genders?



Figure 8.7: Sieve Diagram of Gender vs. Confidence

Sieve Diagram: This is used to show the relation between observed features and a 'null hypothesis' (how well the data matches the expected number of occurrences). A statistically significant association is present ($\chi^2 = 261.66, p < 0.001$), indicating that model confidence is not independent of gender:

- Female Over-representation: A strong positive association (dark blue) exists for confidence ≥ 0.7 .
- Male Under-representation: Conversely, the model is significantly less likely to assign a high confidence (far fewer times than statistically expected).

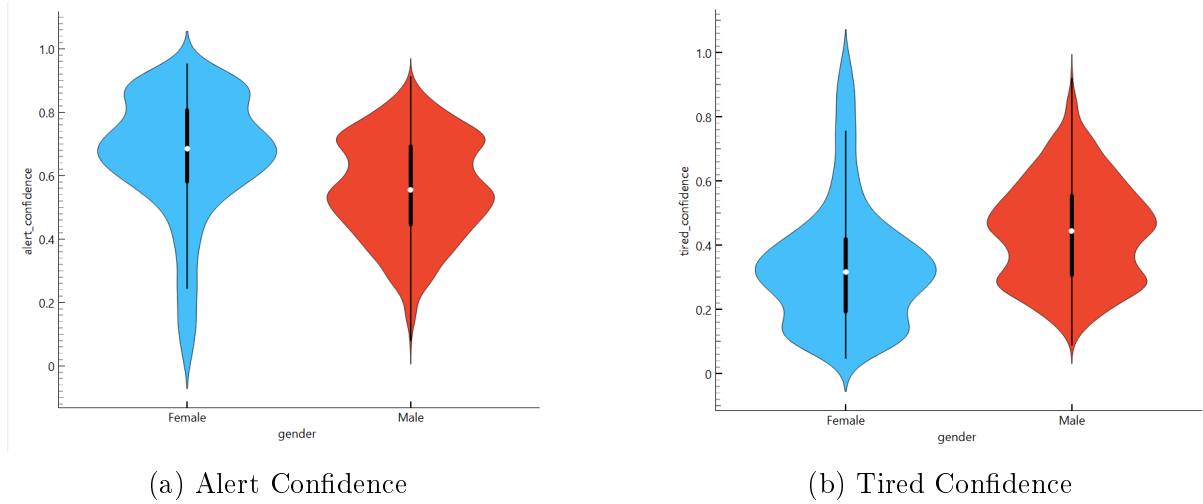


Figure 8.8: Comparison of model confidence scores between genders. Visuallising bias towards Female Alertness

Violin Plot Analysis: While the sieve diagram identifies a frequency anomaly, the violin plots (Figure 8.8) clarify the nature of this bias by visualizing the raw distribution of confidence scores

- **Polarized Certainty: Females** show a higher discriminatory power in terms of their classificaiton, with their **alert_confidence** being skewed more towards the ceiling (1.0) and **tired_confidence** being skewed more towards the floor (0.0). This suggests that the model is more certain of alertness for women.
- **Classification entropy:** For **males**, there is more of a centralised cluster (0.45–0.55). This indicates a higher level of uncertainty in the models predictions, resulting in more lower-conviction evaluations.

Overall, with women being only 12.99% of the total tested dataset, they simply yield a higher level of predictive certainty. This is likely due to higher clarity of training data which could have resulted in overfitting or lack of ambiguity in training data. This could show that the model is more highly-optimised for female-centric characteristics.

8.4.4 Effect of accessories on accuracy

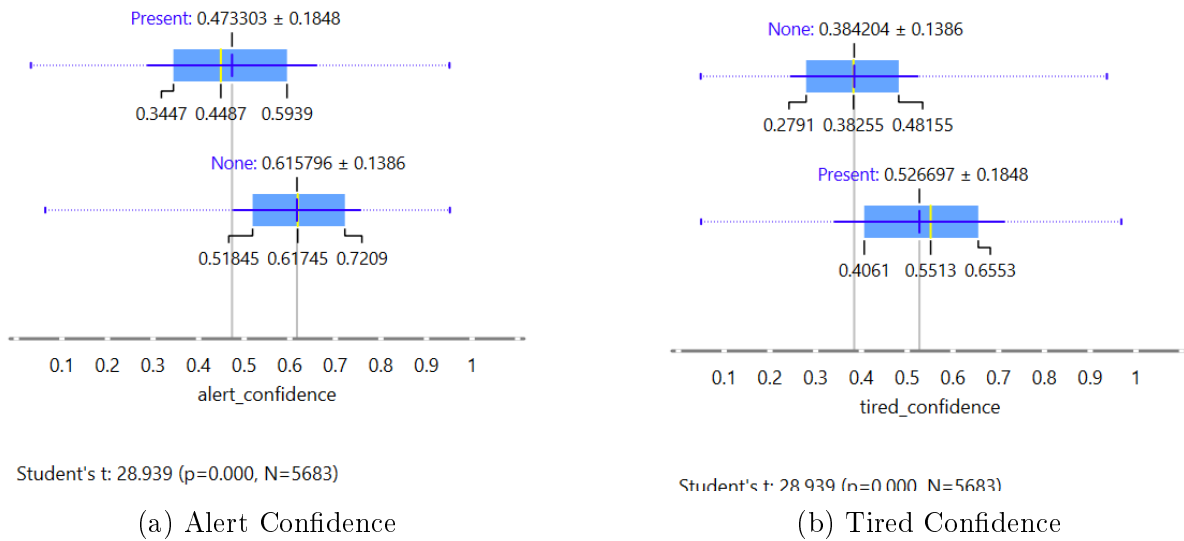


Figure 8.9: Comparison of the effect Accessories have on confidence.

In Figure 8.9, the presence of accessories (such as glasses, hats and masks) appears to have a significant effect on the models confidence. With both graphs showing a significant shift, you can see that the model tends more towards a **tired** classification when accessories are present.

Hypothesis Testing: With the sample size being 1767 images (for the smaller classification – with accessories), the data is sufficient to draw statistically significant conclusions. With a mean difference of 0.142 (14.2%) in alert confidence between the two groups, and a standard deviation of 0.25, the calculated effect size (Cohen's d) is approximately 0.568, which is considered a medium effect size. This indicates that the presence of accessories has a moderate impact on the model's confidence in classifying alertness, with a significant shift towards tired classifications when accessories are present. However, this does suggest that the model works flawlessly with certain images where this is present, but also terribly on specific examples, which is not ideal for a model that is supposed to be versatile and work in a variety of circumstances.

Further Considerations: To further reinforce these findings, I have made use of the **violin** structure in Orange [11] to illustrate this point. If this is true, the main body of the data should sit higher in accessory cases for tired confidence, and lower for alert confidence. This is exactly what we see in the figure below:

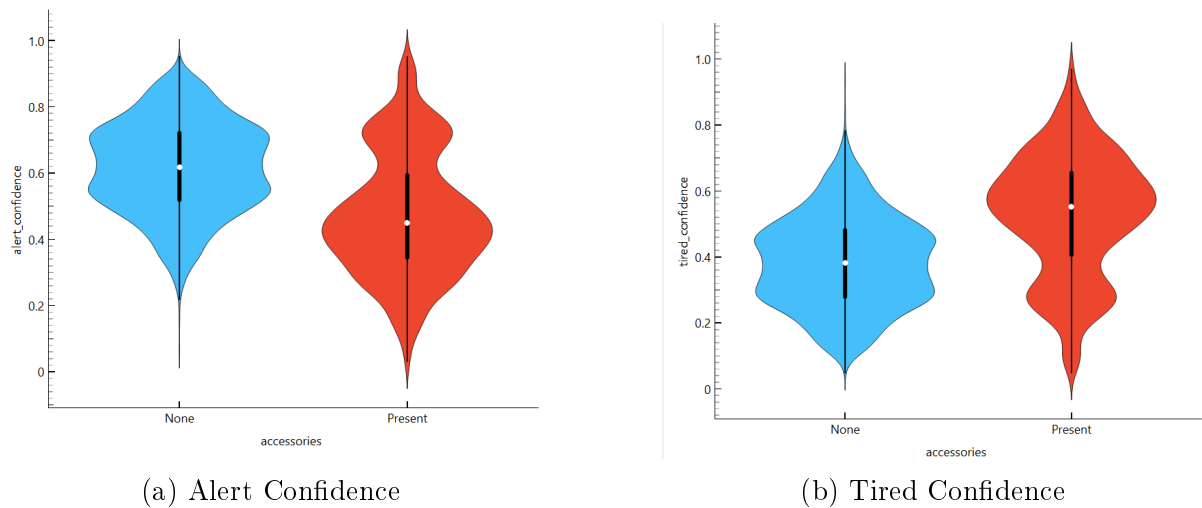


Figure 8.10: Distribution of model confidence scores highlighting that the presence of accessories results in higher confidence for 'Tired' classifications. This shift is clearly visible through the increased distance between the medians in the Tired Confidence subplot (b) compared to the Alert Confidence subplot (a).

8.4.5 How does a user having their hair in their face affect the model?

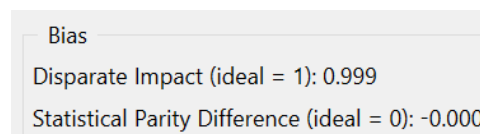


Figure 8.11: Dataset Bias Indication

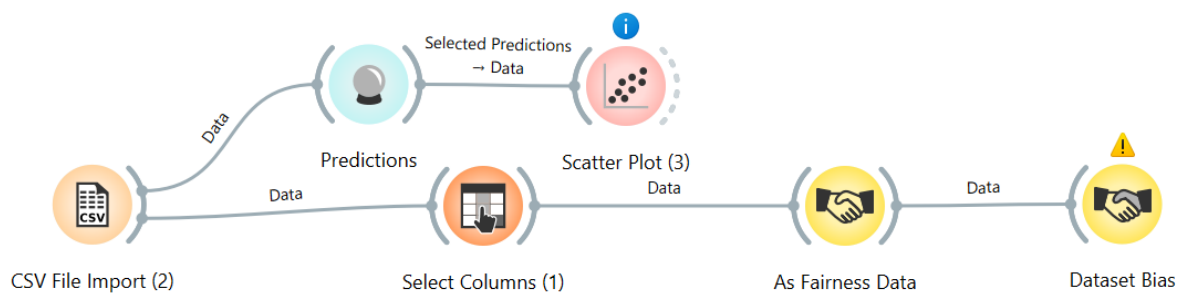


Figure 8.12: Node structure for fairness testing

Note: The warning sign in the Dataset Bias widget refers to missing values within specific feature columns. These issues are omitted from the Disparate Impact calculation to maintain the validity.

With a **Disparate Impact** value of 0.999, there is very little evidence to suggest that the model is unfairly biased to either classification when the subject has their **hair_in_face**. The **Parity Difference** represents the difference in how the model treats the different groups. In this case, there is -0.0 difference, which implies there is a tiny proportion

(uncountably small) tat points to one classificaiton over another. However, this is so small that it could simply represent a single instance which was misslabeled in the entire dataset. Thus implying that the model is 'blind' to the effects of this classificaiton.

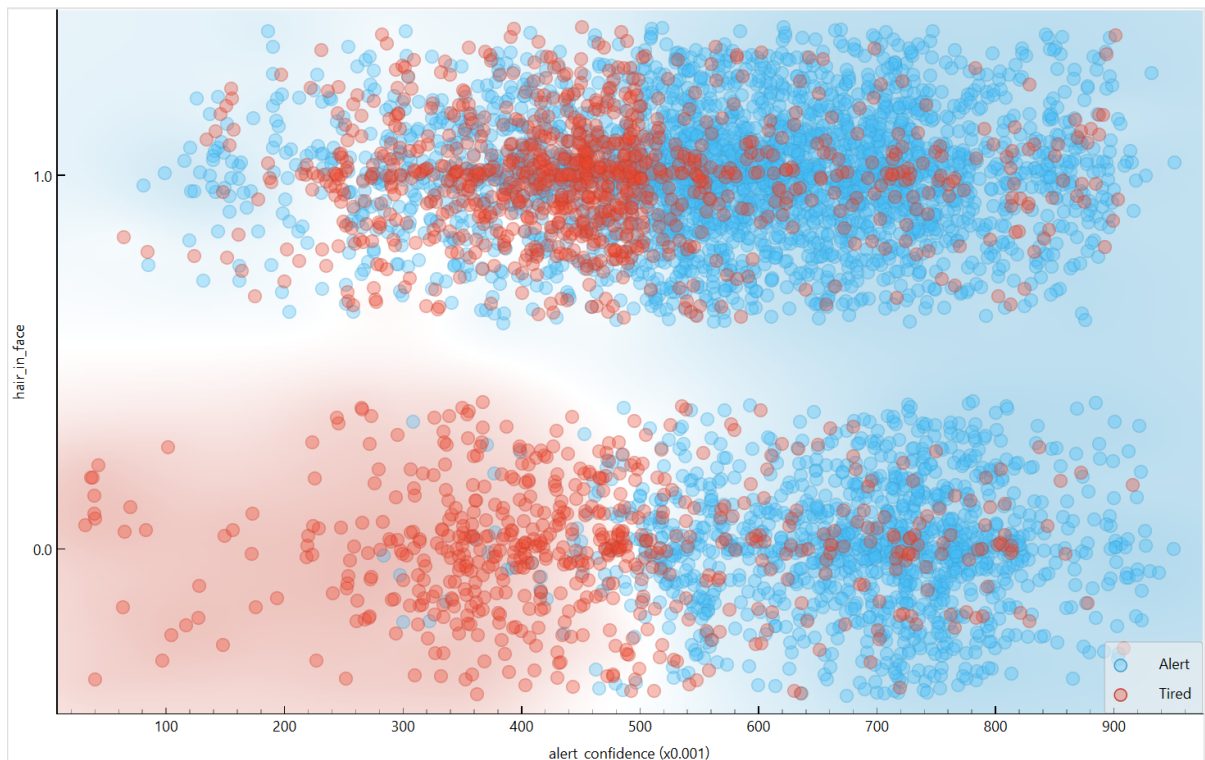


Figure 8.13: Scatter of alert confidence over the binary classifications between hair being in your face and not.

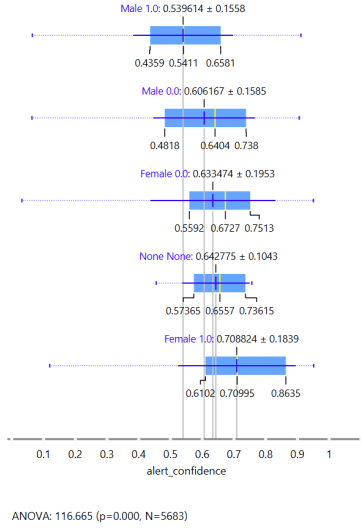
This is re-enforced with the scatter plot where you can see there is no correlation between this and the model's confidence. To account for the linear and unclear nature of dual classifications, the 'Jitter' of the data has been scaled to the maximum to form a maximum disparity between data points to allow for better visualisation.

Hair positioning: Within this classification of **hair_in_face** there are two different representations:

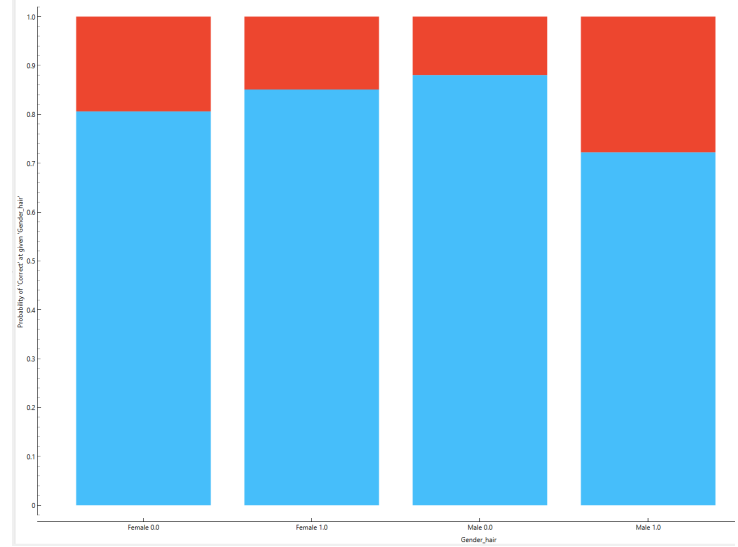
- **Beards** when on a male user
- **hair down** when matched with a female participant

While you can have cross-overs in this representation (e.g. men with long hair that can over their face), through all of the datasets, there was no-one who countered this assumption, so it was easier to classify them as the same (with the distinguishing factor being gender). Since gender bias was already determined in 8.4.3, that women yield a greater predictive certainty, it would be wise to compare the combined effect of these characteristics. Leading to the question:

Is the greater certainty of a classification a result of a greater presence of hair?



(a) Model confidence across subgroups



(b) Classification accuracy distribution

Figure 8.14: Comparison of model confidence and accuracy for Gender and Hair combinations

While the overall data suggests that there is no effect in overall accuracy as a result of a user having their **hair_in_face**, figure 8.14 suggests this is not the case regarding its placement. For male users (where it primarily represents beards), the presence of hair correlates to a decrease in models accuracy, suggesting facial hair acts as **noise** for feature extraction. Conversely, for female participants (representing hair down), the model shows an increase in accuracy and certainty. This suggests that while beards obscure crucial features, long hair inadvertently helps to act as a secondary framing feature, helping the model anchor the face.

8.4.6 Camera position effect on accuracy

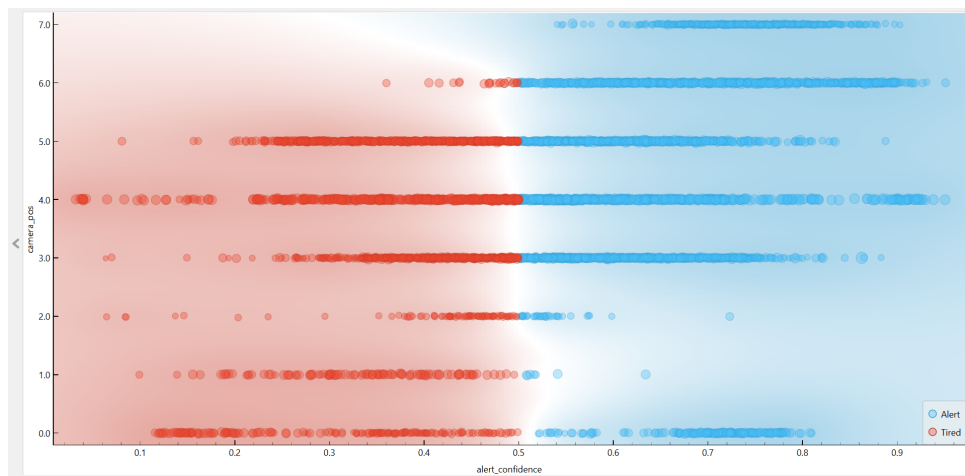


Figure 8.15: Camera Position vs Model Confidence of Classifications

Which represents the positions as follows (as you look towards the driver):

0	1	2
3	4	5
6	7	8

As you can see, the position of the camera has significant effect on accuracy. With higher positions being more likely to be classified as tired (potentially as a result of eyelids being considered more closed with respect to the rest of the eye), while lower positions are more likely to be classified as alert. This is a significant issue as it suggests that the model is not as versatile as I would have hoped, and that the point I initially made about hardware and vehicle versatility is not met.

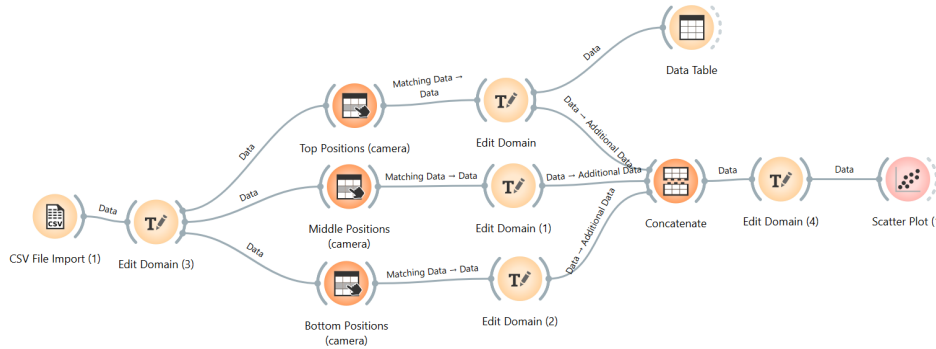


Figure 8.16: Camera Position separation

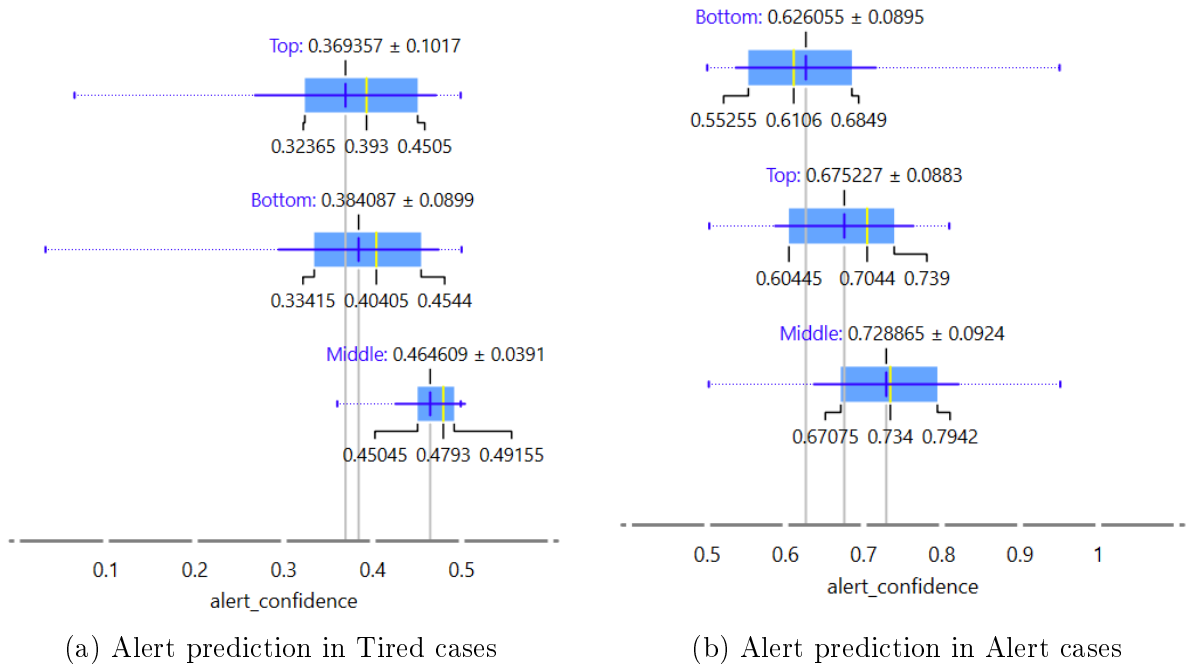


Figure 8.17: Positive predictions across the camera locations

In the Figure above, a distinct correlation between camera position and the model's confidence on True values is evident. The results demonstrate that the model achieves

significantly higher accuracy when the camera is in the Middle position. This aligns with expectations, as the training dataset consists primarily of centered, front-facing imagery.

Hypothesis Testing: Even in the smallest sample group (Top: $N = 587$), the sample size exceeds the minimum requirement of 384 units for statistical significance. We established that a discrepancy greater than 5% between groups would constitute substantial evidence of a "Position Effect." The observed delta between the Middle position (0.46) and the average of the peripheral positions (≈ 0.37) is 0.09 (a 9% difference). Because this 9% gap exceeds our 5% threshold—and given that the 95% confidence intervals for the Middle group do not overlap with the others—there is significant evidence to conclude that camera position has a substantial effect on model performance, with the central position yielding the highest accuracy. High confidence however is only valuable if it correlates to correct classifications.

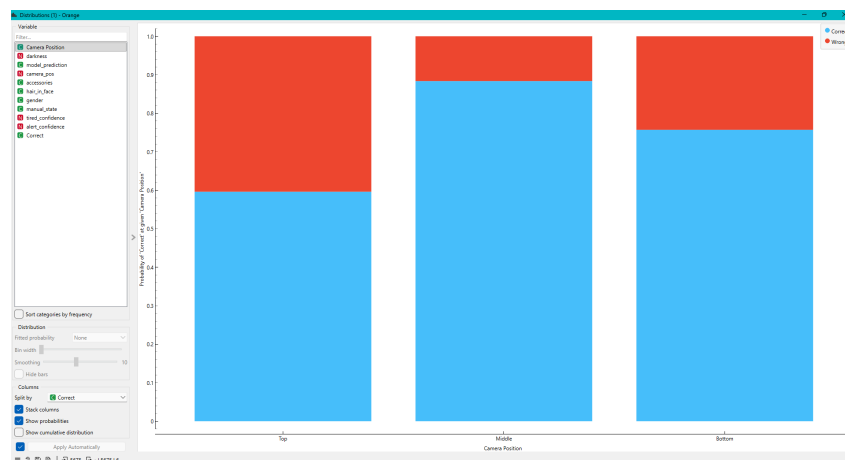


Figure 8.18: proportion bars for correctness over camera positions

As illustrated in Figure 8.18, the model is undoubtedly more accurate (and as shown previously) more certain of a specific classification when the camera is centre mounted, compared to other positions.

8.5 Overall Accuracy

FINAL EVALUATION METRICS	
OVERALL ACCURACY:	77.34%
ALERT Correct:	80.20%
ALERT Wrong:	19.80%
TIRED Correct:	69.76%
TIRED Wrong:	30.24%

Figure 8.19: Accuracy over the Entire Dataset collected

Using the Python node, I counted through every line and counted the differences between the model's prediction and the actual classification. With 50/50 being the average split for simple guesses, independent of any image, a 77% classification is great for a variety of circumstances. Considering that this was all trained off of one dataset, which consisted of a fixed camera angle, same vehicle, same lighting conditions and a limited number of participants with little variation in terms of accessory features, I am more than happy with this final result.

8.5.1 Distribution of Results:

		model_prediction		
	Count	Alert	Tired	Total
manual_state	Alert	3305.0	816.0	4121.0
	Tired	470.0	1084.0	1554.0
	Total	3775.0	1900.0	5675.0

Figure 8.20: counts of correct and incorrect predictions for every combination of predictions

The distribution above illustrates the model's predictive performance across the entire dataset, specifically focusing on the 5,675 complete rows of data. Analysis of the classification errors indicates that the model is more inclined toward False Negative predictions regarding alert states (mistakenly identifying tiredness). From a safety-critical perspective, this bias is preferable; it is more advantageous for the system to err on the side of caution by providing a 'false alarm' than to fail to detect a genuine instance of driver fatigue, which would pose a significantly higher risk.

8.5.2 Confidence in False Cases

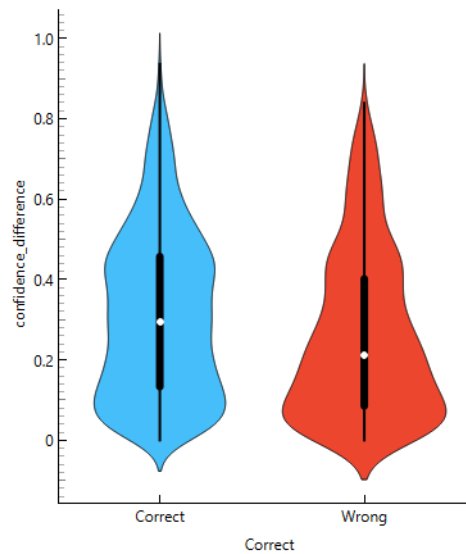


Figure 8.21: Distribution of Peak Classification Confidence (the predicted class) vs Accuracy (right and wrong)

The model demonstrates a desirable "**self-aware**" error profile, where the predictive certainty is reduced in cases where the model is wrong. With Figure 8.21 showing that incorrect classifications exhibit a significantly greater density cluster near the 0.5 mark (border between 2 classes), with correct classifications having a **higher median confidence**. This is statistically supported by the moderate correlation between confidence levels and classification outcomes:

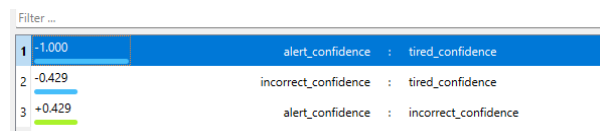


Figure 8.22: Trends between alert and tired confidences and the confidence of the incorrectly determined class

8.5.3 Summary of Attribute Effects and Ideal Environment

To quantify the impact of each variable, the model's performance was measured against the overall baseline accuracy of 77%. The "Delta" represents the percentage point shift in accuracy for this attribute.

Table 8.1: Summary of Attribute Effects on Model Accuracy

Attribute	Acc. Overall	Delta	Key Insight & Ideal Environment
Accessories (None)	83.9%	+6.6%	Presence of eyewear/masks shifts bias toward Alert (91.16%). Sub-optimal for safety-critical environments where false negatives are risky.
Accessories (Present)	62.76%	−14.5%	Superior at predicting Tiredness (83.54%). From a safety standpoint, this "erring on the side of caution" is preferable for fatigue detection.
Gender (Female)	82.2%*	+4.9%	High alert accuracy; however, "Tired" cases show significantly lower parity (44.7%), suggesting potential training set imbalance.
Gender (Male)	76.6%	−0.7%	Highly balanced across classes ($\pm 4.4\%$); significantly less prone to extreme classification bias than female subjects.
Camera (Top)	59.6%*	−17.7%	Over-weighted toward Tired due to downward eyelid perspective; useful specifically for prioritizing safety-critical "false alarms."
Camera (Middle)	88.3%	+11.0%	Optimal for predictive confidence; aligns with the primary training distribution (centered, front-facing imagery).
Camera (Bottom)	75.7%	−1.6%	Most stable and versatile across both classifications with a minimal recorded bias of only 0.16%.
Hair in Face (None)	86.1%	+8.8%	High predictive power but exhibits greater disparity between classes, with a 94.8% bias toward Alert classifications.
Hair in Face (Present)	73.1%	−4.2%	Context-dependent: Facial hair acts as noise for male subjects, while long hair assists in framing for female participants. Minimal accuracy delta of 3.2%.

*Note: High accuracy in these specific subsets is often a result of class-weighted bias rather than general model power.

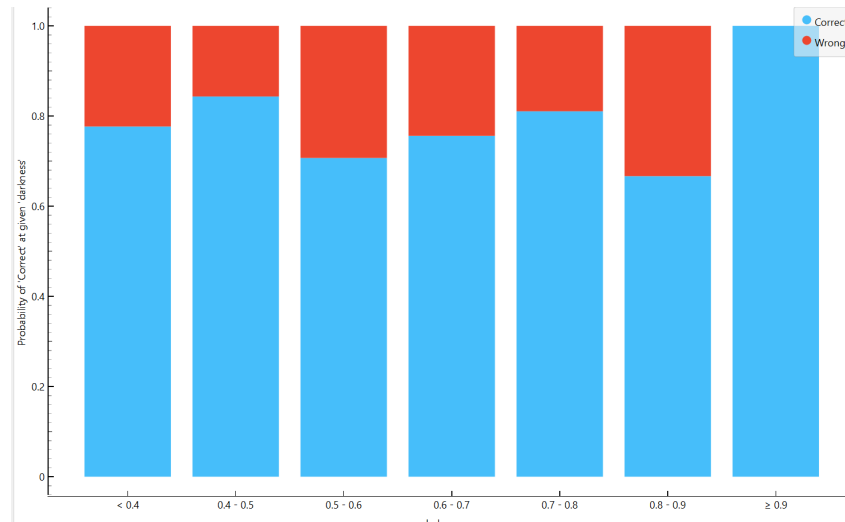


Figure 8.23: Grouped brightness levels and their effect on determining the correct classification

Darkness Distribution There is little correlation between the brightness and the correctness of the model. While it appears that in birgh conditions the model works best, but this is not the case as the ≤ 0.9 only represents 0.04% of the data and so is not representative of the dataset as a whole. The same can be said for < 0.4 where it represents only $\approx 2\%$. As a result, the ideal range is $0.5 \rightarrow 0.8$ as this has a high success rate and has enough data to support this conclusion, as each subsequent section represents around 25% of the data each.

8.6 Ideal Use Case / Environment:

Based on the aggregated data, the "Perfect Environment" for this model is a user with long hair, no facial accessories, situated in a vehicle with a center-bottom mounted camera, and operating in a darkness index between 0.5 and 0.8. Deviations from these parameters—specifically the introduction of eyewear or side-mounted cameras—require a lower confidence threshold for alerts to compensate for the model's inherent biases. Leading to an increased chance of misclassification.

8.7 User Feedback:

get some people to use it, take some pictures of placements and other things
- talk about what they liked/didn't like

8.7.1 Areas of Success

areas of success

8.7.2 Areas of Failure

what could I have done better if done again, or areas that were difficult that really stand out

8.8 Additional Remarks and Development ideas

This section will outline any ideas or areas that I beleive are worth commenting on; with many of these being future considerations given more time.

Conclusion

The final outcome of this project was the development of nine models, each incorporating subtle variations. These models explored both factual, state-based classifications and opinion-driven classifications, where my own judgement was used as a basis for tiredness deduction. This approach introduced challenges during testing, as simulated tiredness behaves differently from genuine tiredness, making it more difficult to create reliable test scenarios throughout development. Despite these limitations, the system achieved an overall accuracy of 77.34%, with a built-in safety bias that favoured misclassifying Alert states as Tired rather than the reverse.

All of this was integrated into a simple, intuitive interface that can be accessed remotely through a convenient web-based platform. The system operates as a headless display, allowing the hardware to remain hidden and unobtrusive while still providing real-time feedback whenever it is needed.

Bibliography

- [1] Brake. (2026). “Driver Fatigue Knowledge Centre.” <https://www.brake.org.uk/get-involved/take-action/mybrake/knowledge-centre/driver-fatigue>
- [2] Brake. (2026). “UK Road Safety Knowledge Centre.” <https://www.brake.org.uk/get-involved/take-action/mybrake/knowledge-centre/uk-road-safety>
- [3] Kavalci Yilmaz, E., and Akcayol, M. A. (2022). “SUST-DDD: A Real-Drive Dataset for Driver Drowsiness Detection.” *Proceedings of the 31st FRUCT Conference*, 31:416–21. doi: 10.5281/zenodo.6519933.
- [4] Ortega, J. D., et al. (2020). “DMD: A Large-Scale Multi-modal Driver Monitoring Dataset for Attention and Alertness Analysis.” *Computer Vision — ECCV 2020 Workshops*, pp. 387–405. Springer International Publishing.
- [5] Yang, C., Yang, Z., Li, W., and See, J. (2023). “FatigueView: A Multi-Camera Video Dataset for Vision-Based Drowsiness Detection.” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 1, pp. 233–246. doi: 10.1109/TITS.2022.3216017.
- [6] UK Government. (2024). “Using a mobile phone while driving.” <https://www.gov.uk/using-mobile-phones-when-driving-the-law>
- [7] Isworo, N., and Liawatimena, S. (2025). “Annotated Drowsiness Detection Dataset Captured Using Raspberry Pi 5.” *Mendeley Data*, V1. doi: 10.17632/chvz7vh2dc.1 <https://data.mendeley.com/datasets/chvz7vh2dc/1>
- [8] Szegedy, C., et al. (2015). “Going deeper with convolutions.” *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1–9.
- [9] Chollet, F. (2017). “Xception: Deep Learning with Depthwise Separable Convolutions.” *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1800–1807.
- [10] Huang, G., et al. (2018). “CondenseNet: An Efficient DenseNet Using Learned Group Convolutions.” *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2752–2761.
- [11] <https://orangedatamining.com/>